

Selective Token-Level Cryptographic Redaction for Privacy-Preserving Clinical Deployment of Large Language Models

Farhan Sheth^{1,*}, Ziyuan Yang^{1,3,*}, Yongying Lan^{1,2}, Si Yong Yeo^{1,3,†}

¹ MedVisAI Lab, Singapore

² Ruijin Hospital, Shanghai Jiao Tong University School of Medicine, China

³ Lee Kong Chian School of Medicine, Nanyang Technological University, Singapore

Abstract

While large language models (LLMs) are increasingly used for clinical applications, many existing pipelines require sending raw sensitive health information to remote servers for processing, which heightens the risk of privacy leakage. A natural approach to mitigate this risk is to encrypt the data before transmission. However, straightforward solutions such as encrypting the entire dataset introduce prohibitive computational, alignment, and communication overheads, rendering large-scale practical deployment infeasible. To preserve privacy while maintaining usability, we present **Healthcare Encryption & Redaction via Adaptive Linguistic Decomposition (HERALD)**, a token-level cryptographic redaction framework designed to achieve this balance by encrypting only sensitive tokens while preserving the surrounding context for downstream model utility. HERALD combines medical named-entity recognizer (NER) with part-of-speech (POS) driven policies to select candidate tokens, performs targeted lemmatization to stabilize surface forms, and substitutes each protected token with a deterministic ciphertext wrapped in explicit delimiters. Notably, HERALD is model-agnostic and operates entirely on the client side, ensuring that sensitive content remains encrypted throughout storage, transmission, and processing without requiring changes to downstream models. We evaluated HERALD on both classification and medical question answering (MQA) tasks on public datasets. Across different tasks, experiments illustrate that fully secured baselines suffer significant utility loss, whereas HERALD consistently recovers performance close to plaintext. Overall, HERALD provides a novel utilization pipeline.

*The first two authors contributed equally to this work.

†Corresponding author: Si Yong Yeo (siyong.yeo@ntu.edu.sg)

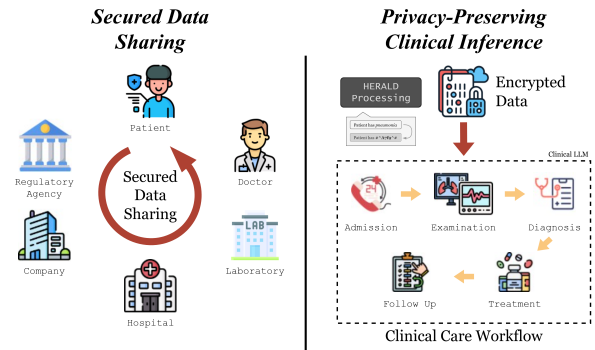


Figure 1. Overview of HERALD for secured clinical data sharing and privacy-preserving clinical deployment of LLMs.

1. Introduction

Internet-scale services have broadened access to medical knowledge, but have also heightened long-standing concerns around data privacy. The rapid integration of large language models (LLMs) in clinical and operational workflows further intensifies these concerns, as LLMs are increasingly used in the clinical prescribing, diagnosis, reporting, and patient communication pipelines [1–5]. Yet in many real-world deployments, private health information (PHI) is still transmitted to remote infrastructure in plaintext, creating risks of privacy leakage through inadvertent logging, adversarial probing, or network interception when interacting with cloud-based services such as ChatGPT [6] and Gemini [7].

In clinical applications such as ICU summarization, discharge instruction generation, and medication reconciliation, raw patient data is often exposed to service providers and intermediary systems, even when transport and storage protections are in place [8]. Hence, transmitting plaintext-based pipeline introduces three classes of risk: (1) **Technical attacks:** LLMs are vulnerable to various threats, including jailbreaks, attribute inference attacks, and others [9–11]. (2) **Policy Limitations:** policies may restrict access to sensitive data, for instance U.S. defense limitations and industry

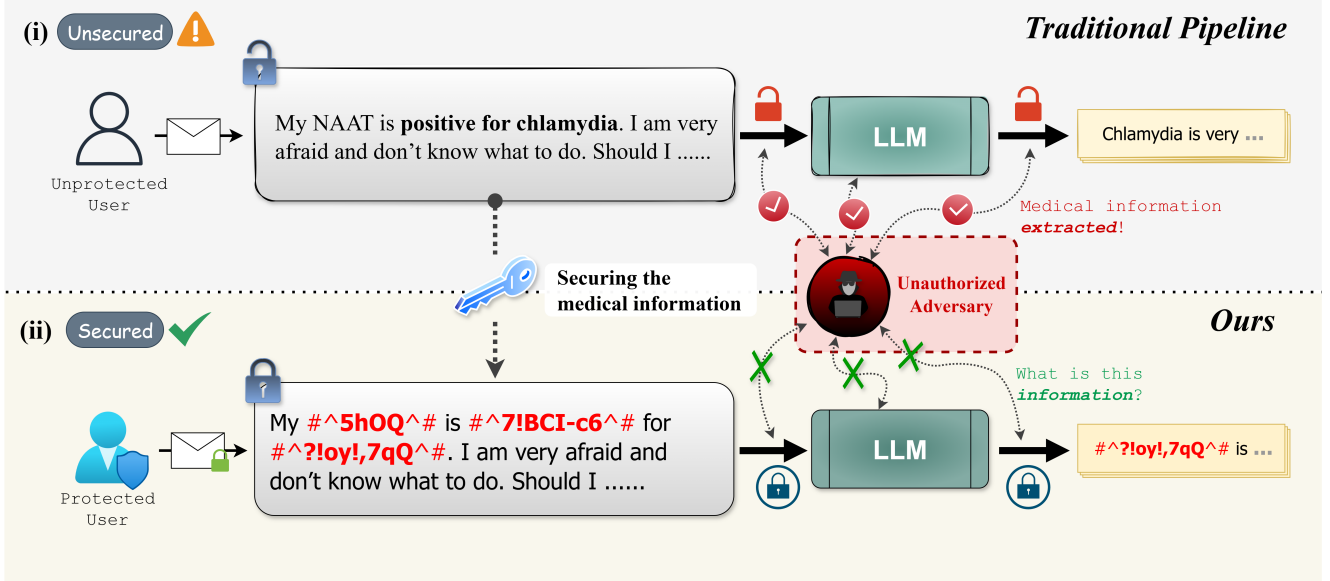


Figure 2. Comparison of HERALD with a traditional pipeline. (i) **Traditional:** plaintext clinical notes are sent to a remote model, exposing sensitive content. (ii) **HERALD:** sensitive spans are encrypted on-device into tokens (shown in red); the server processes only the transformed input and the client decodes outputs locally. Observers see only unreadable strings for protected tokens, while task utility is preserved.

bans motivated by data security concerns.¹ (3) **Regulatory constraints:** The health sector’s requirements complicate scaling, especially when sensitive data routinely crosses organizations [12, 13]. Existing work spans heavyweight cryptography via encrypted computation that evaluates models on ciphertext using homomorphic encryption [14–17]; statistical privacy and secure execution, including secure computation or TEEs for collaborative training or protected inference [18–20]; federated learning as a privacy-preserving approach for biomedical NLP information extraction [21]; and de-identification–style input transformations that mask or restructure tokens or representations [22–27]. However, these defenses often impose substantial computational and communication costs, making them difficult to deploy in real-world clinical settings [24, 25, 28].

To address privacy concerns and ensure practicality, we rethink the pipeline shifting from *sending plaintext to a model* toward privacy-preserving interactions. For medical applications, we observe that the content is largely structured text, which we partition inputs into *sensitive* and *non-sensitive* components. “Sensitive text” includes PHI (names, dates, identifiers), quasi-identifiers, and detailed clinical entities (diagnoses, medications, procedures, labs, narrative findings) while “non-sensitive text” comprises connective language (e.g., articles, prepositions, boilerplate) with minimal identity or clinical risk. Securing only sensitive spans preserves (i) the distributional and syntactic cues in the remaining plaintext and (ii) stable ciphertext token that behave like an auxiliary vocabulary the model can learn to condition

on. This ensures that high-risk spans never appear in the clear during training or inference, and avoids the redundant computation and communication overhead of encrypting non-sensitive content.

Motivated by the above findings, in this paper, we propose a **Healthcare Encryption & Redaction via Adaptive Linguistic Decomposition (HERALD)** method, which is a *token-level* cryptographic redaction framework for client-side preprocessing that is model-agnostic and compatible with local, black-box, and cloud LLMs. Only tokens identified as sensitive are deterministically transformed into keyed ciphertext token identities, wrapped in explicit delimiters; non-sensitive context passes through unchanged. Each sensitive plaintext token maps to the same ciphertext identity, enabling models to learn and reuse representations over encrypted spans without architectural modification. HERALD thus keeps protected content in ciphertext at rest, in transit, and in use, supporting common compliance regimes [12, 13]. Figure 1 summarizes the clinical deployment scenarios of HERALD, which reshapes how clinical data are shared and how LLMs are utilized in clinical practice. Specifically, clinical data are transmitted as ciphertext across different stakeholders, and the inputs to the LLM are also maintained in encrypted form, enabling end-to-end privacy-preserving inference without exposing raw sensitive information.

To clarify how our approach differs from the traditional pipeline, we provide an illustration in Figure 2. Compared with the traditional pipeline that broadly anonymizes, disrupts model-facing structure, requires bespoke tokenization or model changes, and moves plaintext through intermediate systems with fixed privacy settings, our approach: (1)

¹ <https://www.cnbc.com/2025/01/28/us-navy-restricts-use-of-deepseek-ai-imperative-to-avoid-using.html>

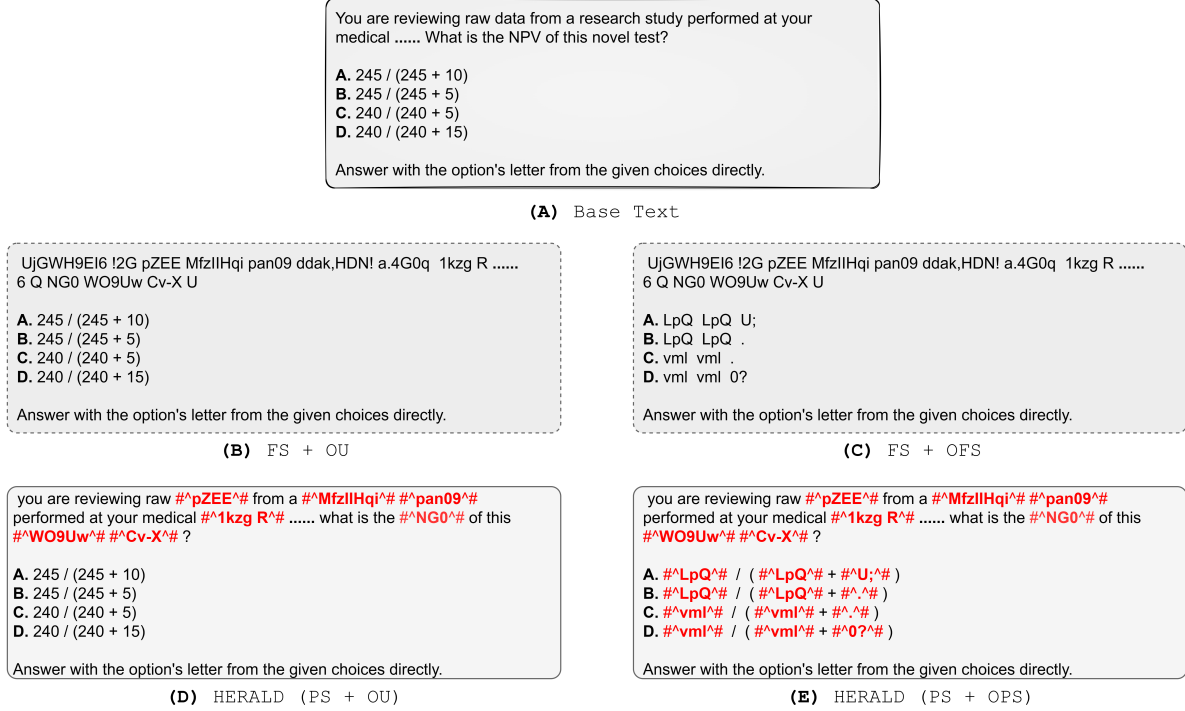


Figure 3. **Text variants used in our MCQ experiments.** (A) Unsecured base multiple-choice question. (B) **FS+OU**: the question stem is fully secured, while the answer options remain in plaintext (“options unsecured”). (C) **FS+OFS**: both stem and options are fully secured. (D) **PS+OU**: the stem is partially secured using HERALD; options remain in plaintext. (E) **PS+OPS**: both stem and options are partially secured with HERALD. These controlled variants allow us to measure model performance under increasing degrees of semantic obfuscation.

protects only what is necessary to limit information loss, (2) preserves learnable structure using stable identities and explicit delimiters, (3) works with standard tokenizers and architectures, (4) enforces end-to-end ciphertext handling, and (5) exposes tunable policies to trade privacy for utility.

The main contributions of this paper can be summarized as follows:

- **Paradigm.** We propose a selective, ciphertext-preserving workflow for clinical NLP: only *sensitive/priority* tokens are deterministically transformed into *stable ciphertext token identities*, while surrounding non-sensitive context remains in plaintext. This enables models to condition jointly on preserved linguistic scaffold and ciphertext, while keeping sensitive content secured end-to-end.
- **Method.** We introduce HERALD, a client-side token-level cryptographic redaction framework. HERALD requires no architectural changes and is compatible with local deployment as well as API-only/black-box and cloud LLM services.
- **Empirical validation and clinical deployment guidance.** We evaluate HERALD on three public medical benchmarks across multiple LLM backbones, reporting both downstream utility and practical costs. The results map an actionable privacy–utility frontier, showing that selective, token-level securing can preserve near-plaintext performance compared to full-token encryption while adding

only modest operational overhead. This directly supports real-world clinical use cases where PHI must remain protected end-to-end (e.g., cloud/API-assisted note drafting and summarization, triage message processing, coding support, and clinician-facing QA) without changing the model or requiring specialized infrastructure.

- **Cipher and tokenization study.** We benchmark a suite of token-level protection primitives under various regimes. We quantify privacy–utility and operational trade-offs in terms of downstream utility and tokenization footprint. This yields concrete, tokenizer-aware guidance for selecting deterministic, deployment-friendly transforms that minimize sequence blow-up while retaining utility, useful for integrating privacy protection into EHR-adjacent NLP pipelines and API-only LLM workflows.

2. Methods

In this section, we describe the protection settings used to evaluate privacy-utility trade-offs and then detail the HERALD framework. Here we define the textual regimes considered in the experiments, including baseline text without protection, fully secured text, and partially secured text generated by HERALD. We then describe the HERALD pipeline, which selectively protects sensitive clinical content through text normalization, targeted base-word transformation, sensitive-token identification, and token-level cryptographic transformation.

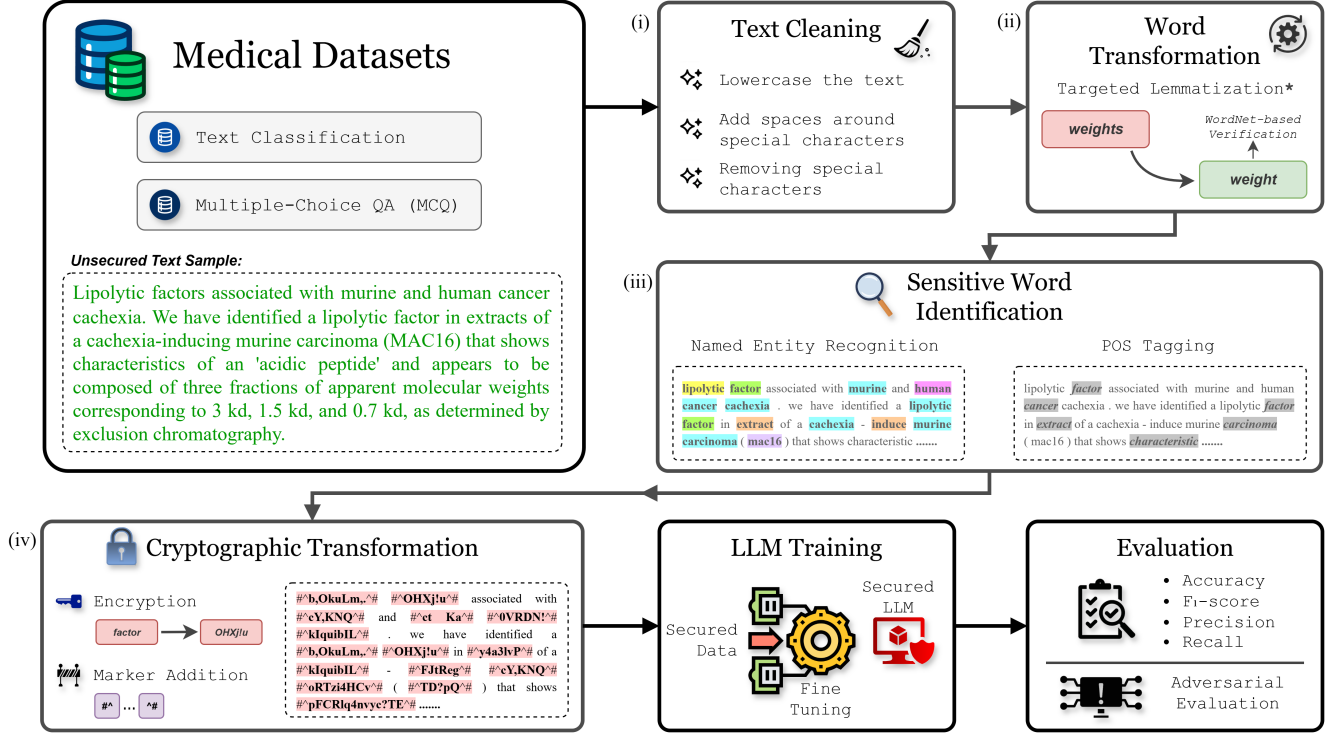


Figure 4. Overview of the HERALD workflow.

2.1. Securing Workflow

We consider three textual variants. The first is a baseline in which the original text is left unchanged and no security or privacy mechanism is applied; this variant provides the reference performance against which our framework is evaluated. The second variant is *fully secured (FS)* text, in which every token is transformed, in contrast to HERALD where only targeted transformations are applied; FS yields the most secure textual form. The third variant is *partially secured (PS)* text produced with the HERALD framework (details in Section 2.2).

Because multiple-choice question (MCQ) datasets include answer options (A, B, C, D), we further distinguish whether the options themselves are transformed in the FS and PS settings. Under FS, options are either *options fully secured (OFS)* or left in their baseline form, i.e., *options unsecured (OU)*. Under PS with HERALD, options are either left unsecured (OU) or *options partially secured (OPS)* via HERALD. Figure 3 shows the baseline alongside four variants (FS+OU, FS+OFS, PS+OU, PS+OPS) on the MCQ dataset (FPE was used for encryption). For the classification dataset, which contains no options, we include the baseline text, FS, and HERALD (PS). Sequence-length inflation and GPU memory implications for these variants are summarized in Appendix A.8.2, with end-to-end preprocessing throughput in Appendix A.8.1.

2.2. HERALD

HERALD is a model-agnostic privacy framework that bridges the gap between heavyweight cryptography and naive text masking through selective token-level protection. Specifically, HERALD follows a four-stage pipeline: (i) text normalization and cleaning, (ii) targeted base-word transformation, (iii) sensitive-token identification, and (iv) token-level cryptographic transformation. Only priority tokens are reduced to their base forms via *targeted lemmatization*, while others are left unchanged to preserve semantics. The end-to-end pipeline is illustrated in Figure 4.

2.2.1. Text Cleaning

The first stage standardizes the input text to stabilize tokenization and remove trivial variability. We lowercase all characters to ensure case-insensitive processing, normalize whitespace around punctuation so symbols are tokenized independently, and remove uncommon special characters. Overall, this yields a clean text x_{clean} that avoids spurious tokens (e.g. casing or punctuation variants) that could hinder downstream learning. Related ablation studies can be found in Appendix B.3.

2.2.2. Word Transformation

Next, HERALD applies **targeted lemmatization** to reduce inflectional sparsity for candidate sensitive tokens. Lemmatization is mapping inflected or derived word forms to their dictionary base (lemma; e.g., *running*→*run*) [29]. So, rather

than lemmatizing every word, which could distort clinically relevant phrasing, we only normalize tokens that are likely to be sensitive and map them to their morphological base forms before cryptographic transformation (using a preliminary sensitive-token detector). For example, “diabetic **patients**” becomes “diabetic **patient**,” “multiple **tumors**” becomes “multiple **tumor**,” and “recent **fractures**” becomes “recent **fracture**.” Similarly, inflected diagnostic terms such as “was **diagnosed** with pneumonia” are reduced to “was **diagnose** with pneumonia” prior to protection.

Because lemmatizers are typically rule-based or statistical, they sometimes return strings that are not valid words in the target lexicon. To filter such cases, we accept a candidate lemma only if it appears in WordNet [30], which organizes English nouns, verbs, adjectives, and adverbs into synonym sets (“synsets”), each representing a lexicalized concept. To improve domain-specific coverage, our methodology supplements WordNet with a curated medical vocabulary. This check guards against nonsensical outputs and over-aggressive lemmatization. Overall, this step produces a partially normalized text x_{base} . By unifying inflected variants of important terms, we improve consistency during cryptographic transformation, thereby aiding the model in learning from secured sensitive data. Crucially, this transformation is **targeted**: only tokens earmarked as sensitive (as determined in the next stage) are lemmatized, which avoids altering common words in a way that might change semantic nuances. We compare targeted vs. untargeted lemmatization and find consistent gains for the targeted strategy (Appendix B.4). Overall, targeted lemmatization approach retains the original sentence structure and meaning for the model, while simplifying sensitive tokens to a canonical form for robust cryptographic transformation.

2.2.3. Sensitive Word Identification

In the third stage, HERALD automatically identifies which tokens carry sensitive information and merit securing. We leverage domain-specific **named entity recognition (NER)** to detect PHI and other medical entities in the text [31]. To protect patient privacy, we de-identify (transform) all PHI and medical data including diagnoses, medications, medical record numbers, and other medical entities. We use MedicalNER, a DeBERTa-based model fine-tuned on PubMed, to detect medical entity types. We augment NER with POS based priority rules to capture additional *content-bearing* tokens that might not be classical named entities but are still important or sensitive in context. Specifically, we treat as sensitive any token that is either (i) a medical entity (e.g. conditions, medications, test results) or (ii) a noun/numeral that is deemed significant to the sentence meaning (ensuring we don’t inadvertently leave an important clue in plaintext). Appendix B.1.1 specifies POS-based tunable privacy tiers for broader sensitive-token detection. For POS tagging, we use the open-source spaCy library [32]. These POS-based rules

act as a backstop so that if the NER misses a rare term, the framework can still flag it due to its grammatical role. Let $\mathcal{I}(x)$ denote the index set of tokens identified as sensitive in input text x after this step. All tokens *not* in $\mathcal{I}(x)$ are considered non-sensitive context and will remain in plaintext. By focusing cryptographic transformation on the most privacy-critical spans [31], HERALD maximizes the protection of patient information while minimizing disruption to the surrounding language that the model needs to interpret.

2.2.4. Cryptographic Transformation

Finally, each sensitive token is transformed via a deterministic cryptographic encryption. We define a cryptographic transformation function $E_k(\cdot)$ parameterized by a secret key k . For each token w_i in the sensitive set $\mathcal{I}(x)$, we first apply the lemmatization function $\ell(w_i)$ (from the previous stage) to obtain its base form, then replace it with a ciphertext string using E_k . We also insert special delimiter markers (denoted by $\langle\langle \dots \rangle\rangle$) around each de-identified (or encrypted) token to explicitly mark its boundaries. Formally, the output token z_i is:

$$z_i = \begin{cases} \langle\langle E_k(\ell(w_i)) \rangle\rangle, & \text{if } i \in \mathcal{I}(x) \\ w_i, & \text{if } i \notin \mathcal{I}(x) \end{cases} \quad (1)$$

The resulting transformed sequence is $T_k(x) = [z_1, z_2, \dots, z_n]$. By construction, tokens outside $\mathcal{I}(x)$ appear verbatim in $T_k(x)$, preserving the surrounding context, while tokens in $\mathcal{I}(x)$ are deterministically mapped (under key k) to stable ciphertext token and wrapped with delimiters. Finally, the tokens in the resulting transformed sequence are concatenated with single spaces to form the final secured sequence. We emphasize that cryptographic transformation (or encryption) is applied at the token level rather than on the entire sequence, aligning with findings that encrypting an entire input string will destroy almost all useful linguistic structure [24].

We use “#^” and “^#” as sentinel tokens (or $\langle\langle \dots \rangle\rangle$) marking the start and end of a cryptographically transformed span, respectively (Related ablation studies can be found in Appendix B.2). These markers help the model in handling these transformed tokens, the inserted markers serve as explicit cues. They delineate the ciphertext boundaries and effectively introduce a simple “syntax” for cryptographically transformed tokens. During fine-tuning, the model learns that any token wrapped in the markers should be treated as an atomic ciphertext lexical item: not human-interpretable and computationally infeasible to invert without the key, yet stable and learnable that the model can embed and condition on across occurrences. This marking prevents the delimiter-wrapped ciphertext strings from being misinterpreted as natural language or split into subwords improperly. It essentially teaches the model a new dialect where sequences inside #^ ... ^# are opaque symbols, each behaving like a sta-

ble, unique word type. Nearest-neighbor recoverability and embedding-space probing indicate cipher tokens do not geometrically ‘point back’ to their plaintext (Appendix C.5, C.4). Therefore, by containing cipher-tokens with special delimiters, we mitigate the risk of the model treating parts of a ciphertext as independent tokens or punctuation.

The outcome of this stage is the final HERALD-protected text. This text can be safely sent to a black-box LLM or stored in a database: it preserves the necessary context and structure for model processing, but any originally sensitive tokens are now “locked” as ciphertext. In summary, HERALD’s workflow keeps non-sensitive context in the clear (for model utility), transforms sensitive tokens into consistent encrypted ciphertext token identities (for privacy), and uses marker cues to ensure the transformed text remains parseable by transformer models during fine-tuning and inference.

2.3. Rationale behind HERALD

Why does HERALD work? HERALD aims to preserve task-relevant linguistic structure while removing direct exposure of high-risk content. Fully securing the entire input largely erases the syntactic and discourse cues that LLMs exploit, leading to substantial utility loss [24]. Conversely, naive masking or deletion often yields ungrammatical inputs and can remain vulnerable to reconstruction from context. By protecting only selected sensitive spans and leaving the remaining context intact, HERALD retains the linguistic scaffold needed for inference while encoding protected spans as ciphertext that function as opaque but learnable lexical items the model can embed and condition on [33].

A key design choice is to replace protected spans with *stable, keyed ciphertext token identities* so the model can accumulate evidence across occurrences and learn representations, similar to how it learns rare lexical types. This improves sample efficiency relative to per-instance randomization or heavy noise, which can fragment supervision and degrade downstream performance [34]. Importantly, HERALD does not rely on an exposed substitution table: ciphertext token identities are computed deterministically from a secret key and are computationally infeasible to invert without it. Practical guessing attacks are therefore limited to cases where the protected domain is intrinsically small and can be exhaustively enumerated; our policies prioritize spans whose candidate space is large (e.g., identifiers, names, free-text entities), and users can increase protection by expanding span selection when auxiliary knowledge is a concern.

From a privacy perspective, HERALD keeps protected spans in ciphertext throughout storage, transmission, and model use, with the secret key remaining client-side. Under standard cryptographic assumptions, observing only the transformed text does not enable efficient recovery of protected values without the key [35]. This reduces exposure even

under remote hosting, logging, or prompt-based elicitation, since any leakage contains ciphertext tokens rather than plaintext [31].

HERALD does not provide information-theoretic secrecy: plaintext context can still support statistical inference about protected spans, particularly under strong auxiliary information. The resulting privacy-utility trade-off is therefore tunable via the span-selection policy, allowing deployments to adopt a conservative setting when inference risk is high and a lighter setting when utility is paramount (Appendix B.5).

2.4. Problem Definition

We formalize the task addressed by HERALD. Let $\mathcal{X}$ denote the space of plaintext inputs (e.g., token sequences in the medical domain) and $\mathcal{Y}$ the output space (e.g., diagnosis labels for classification or free-form answers for QA). For any $x = [w_1, \dots, w_n] \in \mathcal{X}$, a subset of tokens is sensitive. We define a *sensitivity function* $s_n : \mathcal{X}_n \rightarrow 2^{[n]}$ that returns the index set of sensitive tokens in x , where $[n] = 1, \dots, n$ indexes positions in x . The goal is to learn a model $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ (parameters θ) that attains high task performance *without ever observing sensitive tokens in plaintext*. To this end, we introduce a transformation $T_k : \mathcal{X} \rightarrow \mathcal{X}'$, where $\mathcal{X}'$ is the space of partially encrypted texts, with the following properties. First, for any $x \in \mathcal{X}$, $T_k(x)$ deterministically encrypts each sensitive token w_i for $i \in s(x)$ into a ciphertext token while leaving all other tokens unchanged. Second, the transformation is **lossless** for an ideal model in the sense that there exists f_θ whose performance on $T_k(x)$ matches that of the best model on x ; in practice, we approximate this condition by preserving maximal task-relevant information in $T_k(x)$ subject to privacy constraints. Third, encryption is computationally secure: without the secret key k , it is infeasible to recover any sensitive w_i from $T_k(x)$ even when the adversary knows the algorithm T and has unlimited access to samples of encrypted text. This aligns with the proposal that the ciphertext should reveal no information beyond (possibly) length and the indistinguishability-under-chosen-plaintext-attack notion (see Appendix C.2 for jailbreaking and prompt injection attacks and Appendix C.1 for training-data extraction). We note that our practical instantiation may not strictly satisfy such definitions under all tokenization and marker choices; we adopt these ciphers as a deliberate privacy-utility trade-off.

We train f_θ on transformed pairs $(T_k(x^{(j)}), y^{(j)})_{j=1}^N$. The objective is task-specific. For classification, we minimize cross-entropy

$$L(\theta) = -\frac{1}{N} \sum_{j=1}^N \log p_\theta(y^{(j)} | T_k(x^{(j)})) \quad (2)$$

where $p_\theta(y | x)$ denotes the model’s predictive distribution. For generative settings (MCQ), we use a standard language-

modeling negative log-likelihood over target tokens. At inference, user inputs are transformed by the same T_k before they are provided to f_θ , so the model never receives raw sensitive tokens at either training or test time. We assume T_k is invertible only by parties holding k (for deterministic symmetric encryption) or non-invertible (for one-way hashing). Consequently, even if f_θ is compromised or queried adversarially, plaintext sensitive content remains concealed.

Our threat model targets **exposure of plaintext** to unauthorized entities via model leakage or interception. We do not seek to prevent the model from leveraging contextual information or stable ciphertext tokens during learning; rather, we ensure that plaintext sensitive spans are never provided to the model or service. This setting matches common LLM deployments (e.g., cloud APIs) in which the model is an untrusted or partially trusted black-box and the data owner seeks to prevent disclosure of PHI. HERALD enforces that such services observe only encrypted PHI. We further assume the adversary lacks the secret key k and cannot break the underlying cryptography via brute force or cryptanalysis, motivating our use of established primitives (e.g., AES, FPE, or SHA). If the key is compromised (e.g., through system breach or social engineering), encryption offers no protection; key management is therefore essential but out of scope for our NLP focus. Finally, we assume that encryption occurs in a secure environment before fine-tuning so that plaintext is not exposed during training (e.g., local preprocessing or a secure enclave). The trained model itself is not inherently privacy-preserving as memorization could still occur, so HERALD can be combined with complementary defenses such as differentially private fine-tuning [36]. In our experiments, standard fine-tuning on encrypted data did not cause the model to memorize or emit plaintext tokens, as these are never observed, which mitigates the primary memorization risk in practice.

2.5. Theoretical Groundings

From a theoretical perspective, HERALD applies an ε -**cryptographic transformation** to the input text, where ε quantifies information lost through encryption. When $\varepsilon = 0$, the transformation is semantics-preserving for the downstream task; in practice $\varepsilon > 0$ because encrypting a token removes its plaintext identity from the model’s view, replacing it with a stable ciphertext identity, which carry task-relevant signal. Let X denote the random variable over inputs and Y the task label or output. Our aim is to keep the mutual information $I(Y; T_k(X))$ close to $I(Y; X)$ while reducing leakage about sensitive content, formalized as minimizing $I(\text{sensitive}; T_k(X))$, where “sensitive” denotes the sensitive portion of X . Accordingly, HERALD seeks to **maximize** $I(Y; T_k(X))$ (utility) subject to **minimizing** $I(\text{sensitive}; T_k(X))$ (privacy leakage). Under ideal semantic security, ciphertext does not enable efficient recovery of

plaintext from the ciphertext value alone; however, deterministic token-level schemes leak equality/repetition structure. Consequently, any residual inference can exploit both (i) surrounding plaintext context and (ii) repetition/co-occurrence patterns over stable ciphertext identities. Thus, $T_k(X)$ may leak signals beyond context alone (e.g., equality/frequency structure), while still preventing efficient recovery of the exact plaintext token without the secret key. For example, in “The patient’s HIV test came back positive,” if “HIV” is encrypted as $\#^{\wedge}\text{Nco}^{\wedge}\#$, an attacker could still guess the disease from “test came back positive,” a cue present regardless of encryption. An information-theoretic reading would note that $I(\text{disease}; \text{context})$ is high even before transformation; perfect privacy would therefore require sanitizing context as well, which would impair utility. HERALD explicitly opts for **computational privacy**, making recovery of exact token identities cryptographically hard rather than information-theoretic secrecy. This mirrors semantic-security intuitions: while token-level ciphertexts are secure against efficient inversion, we intentionally leave correlated plaintext context, enabling adversaries to do better than chance by exploiting those correlations. Precisely quantifying such leakage reduces to analyzing contextual inference of encrypted token identities (a proxy for attacks that attempt to guess protected spans from remaining plaintext context and ciphertext repetition patterns). If a sensitive token has high conditional entropy given its context, encryption effectively limits exposure; if the token is nearly determined by context, any method that leaves context intact (including HERALD) will inevitably leak. Hence, theoretical protection is strongest for *entropic* identifiers (e.g., IDs, names, irregular strings) and weaker for sensitive attributes tightly coupled to context. In practice, many PHI elements are high-entropy, so encrypting them is effective, whereas strongly signaled medical conditions may require additionally protecting selected context tokens. We empirically bound contextual leakage via (i) semantic-similarity/embedding analyses and (ii) token recoverability stress tests; see Appendix C.4 and Appendix C.5.

A complementary view treats HERALD as introducing a constrained **foreign alphabet** that the model must learn to process. The encrypted spans delimited by markers “ $\#^{\wedge}\dots^{\wedge}\#$ ” act like tokens from an auxiliary “language.” In fine-tuning, the model learns representations for these cipher sequences (or for their constituent subword pieces under the tokenizer), with the markers preventing accidental collisions with ordinary vocabulary. When many distinct sensitive types exist, capacity is allocated across many ciphered forms; nevertheless, the number of categories of sensitive content (e.g., PHI types) is typically far smaller than the full lexicon, and many items such as personal names may be seen only once. When the model encounters an unseen ciphertext form at test time such as $\#^{\wedge}\text{XYZ}^{\wedge}\#$, it behaves like an out-of-vocabulary encrypted token identity: the model cannot lever-

age a previously learned embedding for that specific identity, so it may rely more on surrounding plaintext context to infer coarse role or constraints (e.g., entity type, syntactic function). When ciphertext identities recur across examples, the model can learn representations for them, such as learning a foreign language, enabling consistent conditioning on encrypted spans without recovering plaintext. Because HERALD preserves ample context, predictive performance remains robust, as our experiments indicate. Empirically, models fine-tuned under HERALD exhibit only moderate degradation relative to plaintext training while *significantly outperforming* the naive baseline that encrypts all tokens, which collapses performance. These observations are consistent with prior findings, for example, Mishra *et al.* [24] report that token-level encryption can preserve accuracy across NLP tasks once models are adapted, supporting the view that selective encryption recovers most utility while enabling a configurable security layer.

3. Results

We hypothesized that (i) fully securing every token would erase the contextual and syntactic cues that language models rely on, yielding a pronounced utility drop; (ii) selectively securing only prioritized spans would preserve enough scaffold for models to treat ciphertext as stable learnable information and recover much of the plaintext baseline; and (iii) transforms that better preserve tokenizer stability would enable stronger recovery. Across one classification benchmark and two medical MCQ benchmarks, the experiments support these hypotheses: fully secured variants consistently collapse, whereas HERALD partially secured variants recover substantial performance without architectural changes.

3.1. Language Models

We consider two categories of LLMs: models fine-tuned for supervised medical text classification, and generative LLMs used to address generative datasets, specifically medical multiple-choice questions (MCQs). The former are predominantly encoder-only architectures, with a few decoder-only models included for comparison; most are general-purpose models, complemented by domain-adapted variants tailored to biomedical or clinical text.

These model families map to common stages of medical NLP pipelines that transform unstructured text into actionable signals [5]. Encoder backbones are typically used upstream for de-identification, concept extraction, and document-level classification over clinical notes and biomedical literature [37]. Generative LLMs are increasingly evaluated for clinician-facing question answering and summarization [38, 39], and have begun to enter documentation workflows via ambient systems that draft visit notes from conversations [40].

3.1.1. Classification Models

Classification models are employed as backbones for downstream text classification. The majority are encoder-only Transformer encoders trained with masked language modeling (MLM), while a subset are decoder-only models pretrained with next-token prediction. The used methods include BERT [41], RoBERTa [42], DeBERTa [43], GPT-2 [44], BioBERT [45], ClinicalBERT [46], and BioGPT [47].

3.1.2. Generative Models

We employ generative LLMs to answer medical-domain multiple-choice questions (MCQs). The suite includes both general-purpose LLMs and models adapted to biomedical or clinical text. The used LLMs include Qwen-2.5-7B [48], Mistral-v0.3 [49], Llama-3.1-8B [50], Llama-3.1-Aloe-Beta-8B [51], and Llama-3-Med42-8B [52]. All LLMs are publicly available, and in our experiments we only fine-tune them rather than retraining from scratch.

3.2. Datasets

We select one public dataset for the classification task and two public datasets for the multiple-choice question (MCQ) task. This choice spans two distinct clinical genres: biomedical abstracts and standardized exam questions, thereby testing robustness under heterogeneous lexical and structural distributions. The brief introduction can be found as follows:

Medical Abstracts Text Classification (Med-TC)

Dataset [53]: Med-TC comprises 14,438 medical abstracts labeled into five clinically salient categories: Neoplasms, Digestive System Diseases, Nervous System Diseases, Cardiovascular Diseases, and General Pathological Conditions. The label space covers major disease systems and broad pathology, making the corpus representative of system-level clinical topicality. Its scale (11,550 training abstracts) is adequate for reliably fine-tuning encoder backbones. We follow the authors’ split with 11,550 train and 2,888 test abstracts, reserving 5% of training for validation.

MedMCQA [54]: MedMCQA is a large-scale MCQA dataset constructed from AIIMS and NEET-PG entrance examinations. It contains more than 194k questions spanning approximately 2.4k healthcare topics and 21 medical subjects, with an average question length of 12.77 tokens and substantial topical diversity. The benchmark assesses more than ten reasoning skills across a wide range of medical domains. For computational tractability, we evaluate a curated subset consisting of four-option questions from five core subjects, Medicine, Gynecology & Obstetrics, Pediatrics, Pathology, and Psychiatry, yielding a total of 20,287 items. We construct a subject-stratified split targeting 80/5/15, so that each split preserves the subject mix rather than concentrating on a narrow subset of domains; realized counts are 15,941/995/3,351 for train/validation/test

(78.6%/4.9%/16.5%).

MedQA-USMLE [55]: MedQA-USMLE is a free-form multiple-choice open-domain QA dataset targeting medical problem solving. We restrict analysis to the English portion, containing 12,723 questions derived from the United States Medical Licensing Examination (USMLE). Relative to MedMCQA, its questions more often require multi-step clinical reasoning, making it a complementary stress test for selective encryption when clinically critical entities are obscured. We use the authors’ split of 10,178/1,272/1,273 for train/validation/test.

3.3. Experimental Setup

We evaluate whether token-level protection can meaningfully reduce exposure of sensitive clinical content while preserving downstream task utility. Experiments span two representative clinical NLP workloads: (i) supervised medical text classification and (ii) medical multiple-choice question answering (MCQ), which jointly reflect common settings where models consume patient-linked narratives for categorization or decision support. We compare three privacy regimes: plaintext (*Base*), fully secured text where all tokens are transformed (*FS*), and HERALD partial security where only prioritized tokens are transformed (*PS*). For MCQ prompts, we additionally vary whether answer options remain readable (*OU*) or are transformed (*OFS/OPS*). This protocol directly tests whether selective ciphertext substitution can offer a practical privacy–utility trade-off in healthcare deployments without requiring model or infrastructure changes. We report Accuracy, F_1 -score, Precision, and Recall, and we measure training/inference overhead to assess practical feasibility in healthcare settings.

3.3.1. Cryptographic Methods

To protect sensitive medical text during language model training, we apply token-level transformations. Our baselines span multiple cryptographic families and architectures, but all share a core requirement: the transformation must conceal the original content while preserving sufficient structural or statistical regularities for effective learning. These methods can be roughly divided into encryption-based and obscuration methods.

Encryption-Based Methods. We evaluate symmetric-key ciphers as our baselines, including AES in SIV and ECB modes and Blowfish in ECB mode, as well as format-preserving encryption (FPE). For each method, we summarize theoretical guarantees, known weaknesses, and empirical effects on model performance, thereby characterizing the security-utility trade-off in Appendix A.1.

Obscuration Methods. We evaluate one-way transformations (e.g., hashing) as auxiliary baselines rather than en-

ryption. These methods preserve similarity in the obfuscated space, enabling model generalization, and they support high throughput, deterministic token obfuscation when pre-image resistance is the priority. We study fuzzy hashing, soft hashing, MD5, SHA-1, and SHA-256; details appear in Appendix A.2.

3.3.2. Prompting Strategy

We follow the standard MCQ pipeline in the MMLU style [56]. Each prompt presents the question, options labeled A, B, C, and D, and a short instruction to answer with the option letter only. Prompts are wrapped in the model’s native template, using the official Hugging Face templates for Qwen-2.5, Mistral-v0.3, and Llama-3.1, and the corresponding Llama template for Llama-3.1-Aloe-Beta and Llama-3-Med42.

3.3.3. Evaluation Metrics

We report Accuracy, F_1 -score, Precision, and Recall. Inputs: *Base* (no protection), *FS* (all tokens protected), and HERALD *PS* (only sensitive tokens). For MCQs we also vary option protection: *OU* (unsecured), *OFS* (fully secured), and *OPS* (partially secured via HERALD).

3.3.4. Training Configuration

We attach a linear classification head to each base model and fine-tune with the Hugging Face Trainer. To stabilize optimization and cut computation, a subset of layers is frozen based on model size. All LLMs use unsloth with LoRA adapters. We apply early stopping and a learning-rate scheduler; the final model is the checkpoint with the best validation F_1 . For computational reasons, models are capped at 8B parameters. Cryptographic baselines use a 16-byte key, except AES-SIV which uses 32 bytes. Seed sensitivity is summarized in Appendix A.6, where low coefficients of variation indicate stable results.

3.4. Results and Analysis

3.4.1. Classification Task

For the classification task, performance when trained and evaluated on plaintext is treated as an upper bound (Table 1). BioGPT attains the best accuracy (64.40%), with DeBERTa close behind (63.79%). The small margin between the top models, together with the narrow overall range (57.76%–64.41%), suggests that Med-TC is not dominated by model capacity differences but by the shared ability of strong Transformers to capture abstract-level topical and clinical cues. Notably, domain-adapted models (BioBERT, ClinicalBERT, BioGPT) cluster near the top, indicating that pretraining on biomedical or clinical corpora confers a consistent, albeit modest, advantage in recognizing medically salient lexical and compositional patterns. Accuracy and F_1 track closely across backbones, implying that improvements largely re-

Model	Accuracy (%)	F ₁ -score (%)	Precision (%)	Recall (%)
<i>BERT</i>	61.84	61.40	61.33	61.84
<i>RoBERTa</i>	57.76	57.44	57.32	57.76
<i>DeBERTa</i>	<u>63.79</u>	62.24	<u>63.91</u>	<u>63.79</u>
<i>GPT-2</i>	61.64	60.58	61.00	61.64
<i>BioBERT</i>	63.06	<u>62.80</u>	62.84	63.06
<i>ClinicalBERT</i>	62.54	61.56	62.28	62.54
<i>BioGPT</i>	64.40	63.12	64.45	64.40

Table 1. Baseline classification performance on Med-TC with plaintext data.[†]

flect broad reductions in classification error rather than a precision-heavy or recall-heavy operating point shift.

Table 2 characterizes the privacy-utility trade-off under cryptographic transformations, contrasting Fully Secured (FS), where all tokens are transformed, with HERALD (PS), where only sensitive spans are protected. FS consistently collapses performance because it removes both medical content and the surrounding linguistic scaffold that encoders rely on to disambiguate topic and intent; the remaining signal is largely limited to residual structural artifacts of the transformation and dataset priors. In contrast, PS preserves most non-sensitive context (syntax, discourse glue, and generic clinical framing), allowing models to condition on intact sentence structure while also learning consistent representations for ciphertext identities. This yields systematic utility recovery: part of the signal comes from preserved context, and the other part of the signal is recovered when encrypted types recur and the model can associate those stable ciphertext forms with labels or downstream targets. For example, with FPE on ClinicalBERT, PS improves accuracy from 49.21% to 58.41%, indicating that a substantial fraction of discriminative signal in Med-TC is recoverable using context and high-priority concealed entities.

The residual gap to plaintext remains meaningful (e.g., 58.41% vs. 62.54% for ClinicalBERT), which is expected because Med-TC labels depend directly on specialized entities (diagnoses, procedures, and condition descriptors) that are preferentially encrypted under PS. Differences across cryptographic methods are also diagnostic: approaches that better control tokenization disruption and preserve consistent surface regularities tend to yield higher PS utility. FPE and Fuzzy Hash form the strongest cluster, suggesting that learnability benefits when protected tokens map to representations that remain “tokenizer-friendly” and reduce effective vocabulary fragmentation. Classical hashes, especially SHA-256, trail because their high-entropy, long-form outputs provide little reusable substructure for representation learning, amplifying sparsity under both FS and PS. Across models, ClinicalBERT is frequently among the best under PS, consistent with the hypothesis that in-domain encoders can exploit preserved clinical context more effectively.

[†] **Bold** denotes the column best; underline the second best.

3.4.2. MCQ Task

We evaluate LLMs on clinical MCQ benchmarks. We exclude AES in ECB mode, Soft Hash, and SHA-256 as mechanisms for producing secured-text representations (for MCQ task) because they entail unacceptable security–utility trade-offs in our setting. Collectively, these methods either weaken privacy guarantees (ECB) or sacrifice downstream fidelity (low performance score) and computational efficiency (Soft Hash, SHA-256), making them ill-suited for high-stakes NLP under the evaluation regimes used here. We consider performance (non-generative) metrics for easier and fairer comparison. Because of limited computational resources, generative-metric results are provided in Appendix A.5. The inferred generative results largely recover the baseline, while the fully secured workflow collapses again.

On **MedMCQA** baselines (Table 3), the two domain-adapted Llama variants lead, with Llama-3-Med42-8B at 76.51% accuracy and Llama-3.1-Aloe-Beta-8B at 75.87%. The small 0.65-point gap suggests that, in plaintext, model-specific differences are secondary to shared capacity and domain alignment, whereas the larger separation to general-purpose counterparts (e.g., Qwen-2.5-7B at 71.93% and Llama-3.1-8B at 70.22%) indicates a tangible benefit from biomedical specialization.

Under full security for MedMCQA (Table 4), performance degrades for two distinct reasons tied to prompt semantics. In FS+OU, answer options remain readable, but the stem is entirely transformed, so the model loses the clinical facts needed to discriminate among plausible distractors; the best configuration reaches only 53.76% (Med42 with SHA-1), far below plaintext. In FS+OFS, both stem and options are secured, which removes not only the patient-linked content but also the meaning of the candidate answers themselves; accuracies concentrate near the four-choice chance level, with the best at 39.39% (Qwen with Fuzzy Hash). This separation between OU and OFS indicates that, for MCQ, preserving the option text is not a minor convenience but a primary driver of solvable supervision. Cipher choice still affects outcomes, but the absence of a consistent winner suggests a bottleneck at the level of linguistic utility rather than cryptographic primitive.

HERALD’s partial security on MedMCQA (Table 5) substantially closes the gap by retaining most non-sensitive syntax and discourse while replacing only prioritized tokens with stable ciphertext token identities. The best PS+OU reaches 68.25% (Med42 with FPE), recovering a large portion of the loss observed under FS+OU and approaching the plaintext ceiling within a single-digit margin. In contrast, securing answer options under PS+OPS still incurs a large penalty (68.25% → 57.65% for the same model and cipher), reinforcing that option semantics act as task-critical anchors; once obfuscated, the model must infer the

Cryptographic Methods	Model	FS				HERALD (PS)			
		Accuracy	F ₁ -score	Precision	Recall	Accuracy	F ₁ -score	Precision	Recall
AES (<i>ECB</i>)	<i>BERT-cased</i>	42.64	42.41	42.32	42.64	51.42	50.99	50.69	51.42
	<i>RoBERTa-cased</i>	46.00	45.21	45.25	46.00	54.54	53.53	54.70	54.54
	<i>DeBERTa</i>	43.91	38.76	36.04	43.91	51.94	47.77	45.44	51.94
	<i>GPT-2</i>	43.15	40.85	41.16	43.15	55.51	54.50	54.37	55.51
	<i>BioBERT-cased</i>	47.18	46.68	46.62	47.18	53.84	52.85	52.85	53.84
	<i>ClinicalBERT</i>	46.32	45.84	<u>45.82</u>	46.32	57.20	56.53	<u>56.38</u>	57.20
	<i>BioGPT</i>	46.50	46.22	46.85	46.50	<u>56.34</u>	<u>56.23</u>	56.79	56.34
AES (<i>SIV</i>)	<i>BERT-cased</i>	42.06	41.77	41.62	42.06	52.60	51.70	52.26	52.60
	<i>RoBERTa-cased</i>	46.77	46.46	47.15	46.77	53.67	53.29	53.35	53.67
	<i>DeBERTa</i>	44.35	42.15	45.03	44.35	51.04	48.18	48.95	51.04
	<i>GPT-2</i>	43.79	43.12	43.83	43.79	54.05	53.71	53.81	54.05
	<i>BioBERT-cased</i>	45.56	45.27	45.65	45.56	52.87	52.73	52.99	52.87
	<i>ClinicalBERT</i>	46.56	46.07	46.73	46.56	56.89	56.41	56.29	56.89
	<i>BioGPT</i>	45.48	44.93	45.24	45.48	53.81	53.41	53.64	53.81
Blowfish (<i>ECB</i>)	<i>BERT-cased</i>	44.50	44.06	43.79	44.50	56.09	55.44	55.14	56.09
	<i>RoBERTa-cased</i>	46.83	46.55	46.98	46.83	55.71	54.91	56.10	55.71
	<i>DeBERTa</i>	42.62	37.61	34.99	42.62	54.47	52.21	<u>53.48</u>	54.47
	<i>GPT-2</i>	43.15	41.79	42.57	43.15	53.88	51.64	53.06	53.88
	<i>BioBERT-cased</i>	46.53	45.94	45.72	46.53	54.50	53.74	54.27	54.50
	<i>ClinicalBERT</i>	48.62	47.78	48.00	48.62	57.34	56.29	56.11	57.34
	<i>BioGPT</i>	47.41	46.74	46.76	47.41	<u>56.13</u>	55.15	55.60	56.13
FPE	<i>BERT-cased</i>	46.47	45.63	45.81	46.47	56.23	55.17	55.08	56.23
	<i>RoBERTa-cased</i>	46.24	45.81	45.86	46.24	53.95	53.54	53.85	53.95
	<i>DeBERTa</i>	43.38	38.18	34.77	43.38	52.46	47.12	49.01	52.46
	<i>GPT-2</i>	48.65	47.90	47.78	48.65	57.86	57.24	57.01	57.86
	<i>BioBERT-cased</i>	44.68	43.98	44.23	44.68	56.23	55.86	56.21	56.23
	<i>ClinicalBERT</i>	49.21	48.36	48.23	49.21	58.41	57.50	57.43	58.41
	<i>BioGPT</i>	47.45	47.04	47.12	47.45	57.65	57.07	56.96	57.65
Fuzzy Hash	<i>BERT-cased</i>	47.85	47.59	47.50	47.85	56.16	55.55	56.34	56.16
	<i>RoBERTa-cased</i>	48.42	47.97	47.77	48.42	53.22	52.52	52.77	53.22
	<i>DeBERTa</i>	47.57	45.81	46.61	47.57	53.60	53.13	53.75	53.60
	<i>GPT-2</i>	48.27	47.38	47.23	48.27	56.48	55.17	55.27	56.48
	<i>BioBERT-cased</i>	48.12	47.33	47.18	48.12	55.78	54.60	54.73	55.78
	<i>ClinicalBERT</i>	49.44	48.93	48.86	49.44	58.21	56.90	57.00	58.21
	<i>BioGPT</i>	47.33	46.78	47.31	47.33	54.74	53.91	53.87	54.74
Soft Hash	<i>BERT-cased</i>	41.79	36.82	34.55	41.79	42.76	42.12	42.26	42.76
	<i>RoBERTa-cased</i>	44.91	44.27	44.94	44.91	51.28	50.68	50.98	51.28
	<i>DeBERTa</i>	36.99	32.31	29.90	36.99	49.76	43.80	44.92	49.76
	<i>GPT-2</i>	39.38	35.09	40.24	39.38	52.04	50.82	51.89	52.04
	<i>BioBERT-cased</i>	42.91	38.90	38.69	42.91	47.58	46.36	46.69	47.58
	<i>ClinicalBERT</i>	41.11	35.92	33.06	41.11	56.51	55.35	56.10	56.51
	<i>BioGPT</i>	45.36	44.84	<u>44.77</u>	45.36	46.81	43.95	47.67	46.81
MD5	<i>BERT-cased</i>	46.77	46.38	46.30	46.77	52.18	51.74	51.51	52.18
	<i>RoBERTa-cased</i>	46.65	46.09	46.06	46.65	55.12	54.79	55.31	55.12
	<i>DeBERTa</i>	43.15	37.89	34.99	43.15	51.49	47.99	49.90	51.49
	<i>GPT-2</i>	44.27	42.33	43.40	44.27	51.47	47.98	49.91	51.47
	<i>BioBERT-cased</i>	47.15	46.78	47.53	47.15	56.65	56.08	56.24	56.65
	<i>ClinicalBERT</i>	47.35	46.55	46.69	47.35	57.69	57.04	57.30	57.69
	<i>BioGPT</i>	47.95	47.71	47.63	47.95	56.41	55.77	55.68	56.41
SHA-1	<i>BERT-cased</i>	42.06	41.54	41.27	42.06	51.35	50.89	50.64	51.35
	<i>RoBERTa-cased</i>	46.83	45.93	46.52	46.83	<u>57.06</u>	56.46	56.67	57.06
	<i>DeBERTa</i>	42.76	37.62	34.67	42.76	50.93	46.21	<u>45.60</u>	50.93
	<i>GPT-2</i>	45.53	44.84	45.02	45.53	54.02	52.46	54.28	54.02
	<i>BioBERT-cased</i>	45.92	45.42	45.81	45.92	53.25	52.35	52.38	53.25
	<i>ClinicalBERT</i>	47.27	46.50	46.44	47.27	57.69	56.95	56.96	57.69
	<i>BioGPT</i>	47.71	47.42	48.03	47.71	55.82	55.52	55.54	55.82
SHA-256	<i>BERT-cased</i>	39.47	38.68	39.82	39.47	45.67	43.61	45.66	45.67
	<i>RoBERTa-cased</i>	39.12	34.59	36.30	39.12	50.31	49.98	51.18	50.31
	<i>DeBERTa</i>	37.23	32.30	31.79	37.23	40.69	35.59	34.96	40.69
	<i>GPT-2</i>	37.91	33.60	39.70	37.91	49.10	46.34	48.40	49.10
	<i>BioBERT-cased</i>	40.55	40.08	41.62	40.55	49.97	49.14	50.52	49.97
	<i>ClinicalBERT</i>	40.77	36.44	37.24	40.77	49.76	46.22	53.75	49.76
	<i>BioGPT</i>	39.30	38.28	<u>40.20</u>	39.30	48.16	47.38	48.14	48.16

Table 2. **Classification under cryptographic transformations.** Comparison of Fully Secured (FS) vs. HERALD (Partially Secured, PS) settings across ciphers.[†]

correct label without access to the candidate answers’ meaning, which predictably increases ambiguity. Across ciphers, PS+OU consistently exceeds PS+OPS, and the strongest re-

sults cluster around FPE and Fuzzy Hash. These transforms better preserve surface regularities and tokenization stability than high-entropy encodings. Model rankings also partially

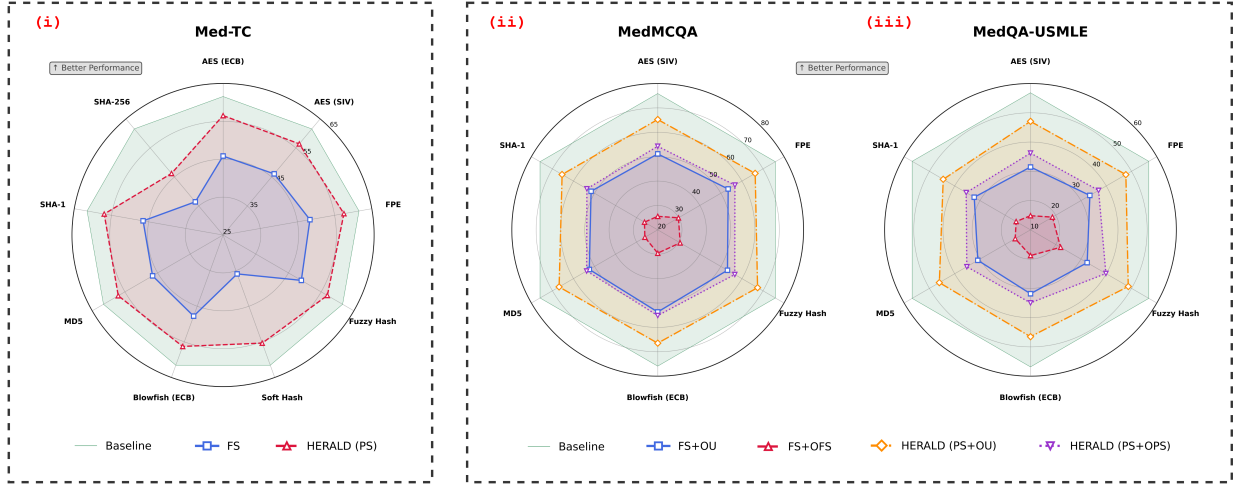


Figure 5. **Utility under full vs. selective security.** Radar plots report test F_1 for *ClinicalBERT*/Med-TC, *Llama-3.1-Aloe-Beta-8B*/MedMCQA, and *Qwen-2.5-7B*/MedQA-USMLE. The outer green ring is plaintext; inner polygons compare FS (all tokens secured) with HERALD (PS). For MCQ, options are either plaintext (OU) or secured (OFS/OPS).

Model	Accuracy (%)	F_1 -score (%)	Precision (%)	Recall (%)
<i>Qwen-2.5-7B</i>	71.93	71.94	72.08	71.93
<i>Mistral-7B-Instruct-v0.3</i>	63.59	63.50	63.56	63.59
<i>Llama-3.1-8B</i>	70.22	70.20	70.26	70.22
<i>Llama-3.1-Aloe-Beta-8B</i>	75.87	75.89	75.94	75.87
<i>Llama-3-Med42-8B</i>	76.51	76.52	76.57	76.51

Table 3. Baseline performance on MedMCQA with plaintext data.[†]

re-emerge under PS+OU: medically adapted Llamas regain clear advantages over the general baselines, indicating that domain knowledge remains usable when the prompt retains readable structure, even if sensitive spans are concealed.

For **MedQA-USMLE** baselines (Table 6), the leading models are separated by less than one point (*Qwen-2.5-7B* at 56.95% vs. *Llama-3.1-Aloe-Beta-8B* at 56.25%), and all models concentrate in the mid-50s. This compression suggests that MedQA-USMLE is dominated by a dataset-level difficulty ceiling rather than large capacity gaps, with most backbones converging to similar error profiles. In contrast to MedMCQA, biomedical adaptation does not uniformly translate into a clear advantage here (e.g., Med42 trails despite domain specialization), consistent with an exam-style setting where multi-step reasoning and distractor discrimination limit gains from surface domain recall alone.

Full security on MedQA-USMLE (Table 7) induces a markedly different regime than plaintext. In FS+OU, options remain readable but the stem is fully transformed, so models can only exploit superficial option priors and residual formatting cues; this yields a plateau in the low-to-mid 30s, with limited separation across cryptographic methods and backbones. In FS+OFS, both stem and options are secured, which removes not only patient-linked content but also the meaning of the candidate answers; accuracies con-

centrate near the four-choice chance level ($\approx 25\%$), indicating that supervision becomes weakly informative once the label space is obfuscated and, in extreme cases, degenerate decoding (e.g., Med42 at 0.00% under Blowfish and SHA-1). Differences across ciphers become second-order under this regime: even comparatively tokenizer-friendly transforms cannot compensate for the removal of interpretable anchors.

HERALD on MedQA-USMLE (Table 8) recovers utility by preserving the syntactic and discourse scaffold that supports exam-style reasoning while learning representations of protected spans as interpretable information. Under PS+OU, top results concentrate around ≈ 49 -50%, which indicates that a substantial fraction of MedQA signal is accessible from problem structure, relations, and non-sensitive descriptors even when prioritized entities are concealed. This recovery is not uniform across settings: ciphers that better preserve tokenizer-friendly regularities (notably FPE and Fuzzy Hash) tend to occupy the top of the PS+OU band. In contrast, PS+OPS consistently underperforms PS+OU, reflecting that option semantics function as anchors for mapping inferred clinical states to discrete choices; once the option surface forms are secured, the model must select among ciphertext labels with reduced semantic grounding, increasing ambiguity and amplifying minor prompt priors.

Figure 5 shows radar plots of F_1 -score across cryptographic transforms: (i) Med-TC with *ClinicalBERT*, (ii) MedMCQA with *Llama-3.1-Aloe-Beta-8B*, and (iii) MedQA-USMLE with *Qwen-2.5-7B* (outer ring = plaintext).

Global synthesis. Across both classification and MCQ tasks, a consistent pattern emerges: fully secured workflows impose steep utility costs, while HERALD’s selective partial securing recovers substantial accuracy without architectural

Cryptographic Methods	Model	FS + OU				FS + OFS			
		Accuracy	F ₁ -score	Precision	Recall	Accuracy	F ₁ -score	Precision	Recall
AES (<i>SIV</i>)	<i>Qwen-2.5-7B</i>	50.53	50.22	50.35	50.53	36.14	29.93	34.97	36.14
	<i>Mistral-7B-Instruct-v0.3</i>	37.08	34.21	37.05	37.08	36.38	25.97	37.28	36.38
	<i>Llama-3.1-8B</i>	45.82	45.77	45.83	45.82	29.13	28.88	28.74	29.13
	<i>Llama3.1-Aloe-Beta-8B</i>	51.18	51.15	51.33	51.18	35.72	25.55	25.54	35.72
	<i>Llama3-Med42-8B</i>	53.47	53.36	53.57	53.47	37.03	29.87	36.47	37.03
Blowfish (<i>ECB</i>)	<i>Qwen-2.5-7B</i>	51.05	50.86	50.90	51.05	37.00	31.72	35.97	37.00
	<i>Mistral-7B-Instruct-v0.3</i>	35.69	31.79	34.84	35.69	36.20	25.72	31.14	36.20
	<i>Llama-3.1-8B</i>	44.88	44.70	44.63	44.88	30.14	29.72	29.49	30.14
	<i>Llama3.1-Aloe-Beta-8B</i>	52.70	52.68	52.82	52.70	36.88	29.56	34.77	36.88
	<i>Llama3-Med42-8B</i>	52.84	52.76	52.85	52.84	37.96	31.04	36.38	37.96
FPE	<i>Qwen-2.5-7B</i>	52.00	51.82	51.88	52.00	36.77	31.13	33.88	36.77
	<i>Mistral-7B-Instruct-v0.3</i>	37.58	35.22	36.78	37.58	36.94	28.63	34.02	36.94
	<i>Llama-3.1-8B</i>	39.47	39.29	39.21	39.47	30.68	30.40	30.24	30.68
	<i>Llama3.1-Aloe-Beta-8B</i>	53.55	53.48	53.53	53.55	36.79	29.86	35.66	36.79
	<i>Llama3-Med42-8B</i>	53.10	53.06	53.28	53.10	38.59	32.82	38.19	38.59
Fuzzy Hash	<i>Qwen-2.5-7B</i>	52.76	52.65	52.71	52.76	39.39	36.05	38.11	39.39
	<i>Mistral-7B-Instruct-v0.3</i>	38.82	36.66	38.72	38.82	37.93	30.84	38.19	37.93
	<i>Llama-3.1-8B</i>	43.65	43.27	43.21	43.65	30.77	30.47	30.27	30.77
	<i>Llama3.1-Aloe-Beta-8B</i>	53.18	53.14	53.28	53.18	37.60	30.66	36.09	37.60
	<i>Llama3-Med42-8B</i>	53.60	53.56	53.64	53.60	37.46	31.94	35.10	37.46
MD5	<i>Qwen-2.5-7B</i>	50.38	50.33	50.51	50.38	35.89	29.68	33.17	35.89
	<i>Mistral-7B-Instruct-v0.3</i>	38.00	35.41	37.89	38.00	36.35	26.70	38.61	36.35
	<i>Llama-3.1-8B</i>	45.57	45.38	45.30	45.57	29.72	29.21	29.00	29.72
	<i>Llama3.1-Aloe-Beta-8B</i>	52.54	52.48	52.62	52.54	35.96	26.10	32.33	35.96
	<i>Llama3-Med42-8B</i>	53.18	53.08	53.36	53.18	36.81	27.60	33.57	36.81
SHA-1	<i>Qwen-2.5-7B</i>	49.68	49.55	49.73	49.68	35.83	27.31	32.46	35.83
	<i>Mistral-7B-Instruct-v0.3</i>	35.76	31.77	35.20	35.76	36.02	25.84	42.19	36.02
	<i>Llama-3.1-8B</i>	43.99	43.96	43.93	43.99	30.23	29.70	29.45	30.23
	<i>Llama3.1-Aloe-Beta-8B</i>	51.68	51.64	51.84	51.68	35.96	26.29	34.69	35.96
	<i>Llama3-Med42-8B</i>	53.76	53.73	54.00	53.76	36.30	27.01	34.58	36.30

Table 4. Performance on MedMCQA with fully secured tokens.[†]

Cryptographic Methods	Model	HERALD (PS + OU)				HERALD (PS + OPS)			
		Accuracy	F ₁ -score	Precision	Recall	Accuracy	F ₁ -score	Precision	Recall
AES (<i>SIV</i>)	<i>Qwen-2.5-7B</i>	63.21	63.14	63.17	63.21	50.76	50.32	50.58	50.76
	<i>Mistral-7B-Instruct-v0.3</i>	51.69	51.01	51.34	51.69	41.39	37.19	41.40	41.39
	<i>Llama-3.1-8B</i>	58.07	57.99	57.97	58.07	45.27	44.83	44.67	45.27
	<i>Llama3.1-Aloe-Beta-8B</i>	65.29	65.25	65.50	65.29	54.61	54.27	54.51	54.61
	<i>Llama3-Med42-8B</i>	67.23	67.16	67.23	67.23	56.13	55.81	55.94	56.13
Blowfish (<i>ECB</i>)	<i>Qwen-2.5-7B</i>	65.09	65.01	65.18	65.09	54.79	54.38	54.42	54.79
	<i>Mistral-7B-Instruct-v0.3</i>	51.12	50.28	50.79	51.12	43.15	40.29	42.98	43.15
	<i>Llama-3.1-8B</i>	60.97	60.86	60.84	60.97	43.24	42.93	42.83	43.24
	<i>Llama3.1-Aloe-Beta-8B</i>	66.55	66.48	66.64	66.55	55.48	55.17	55.23	55.48
	<i>Llama3-Med42-8B</i>	66.99	66.92	67.05	66.99	56.01	55.68	55.81	56.01
FPE	<i>Qwen-2.5-7B</i>	63.12	63.06	63.20	63.12	54.94	54.65	54.77	54.94
	<i>Mistral-7B-Instruct-v0.3</i>	53.00	52.45	52.80	53.00	44.17	41.91	44.18	44.17
	<i>Llama-3.1-8B</i>	59.80	59.80	59.83	59.80	50.43	50.08	50.01	50.43
	<i>Llama3.1-Aloe-Beta-8B</i>	66.40	66.30	66.48	66.40	56.97	56.67	56.92	56.97
	<i>Llama3-Med42-8B</i>	68.25	68.24	68.38	68.25	57.65	57.48	57.48	57.65
Fuzzy Hash	<i>Qwen-2.5-7B</i>	64.94	64.90	65.02	64.94	55.83	55.58	55.70	55.83
	<i>Mistral-7B-Instruct-v0.3</i>	51.92	51.41	51.70	51.92	44.73	42.79	44.00	44.73
	<i>Llama-3.1-8B</i>	61.15	61.11	61.12	61.15	49.36	48.96	48.96	49.36
	<i>Llama3.1-Aloe-Beta-8B</i>	67.44	67.43	67.65	67.44	56.73	56.61	56.64	56.73
	<i>Llama3-Med42-8B</i>	60.19	56.14	58.95	60.19	52.69	49.21	50.00	52.69
MD5	<i>Qwen-2.5-7B</i>	62.10	62.03	62.07	62.10	51.39	50.79	51.30	51.39
	<i>Mistral-7B-Instruct-v0.3</i>	51.69	51.01	51.45	51.69	43.99	41.64	43.47	43.99
	<i>Llama-3.1-8B</i>	60.55	60.47	60.58	60.55	46.67	46.32	46.23	46.67
	<i>Llama3.1-Aloe-Beta-8B</i>	66.88	66.82	67.13	66.88	54.22	53.73	54.31	54.22
	<i>Llama3-Med42-8B</i>	67.89	67.91	68.13	67.89	54.58	54.27	54.53	54.58
SHA-1	<i>Qwen-2.5-7B</i>	61.06	61.02	61.07	61.06	48.76	47.97	48.41	48.76
	<i>Mistral-7B-Instruct-v0.3</i>	52.49	51.90	52.22	52.49	45.66	44.43	45.04	45.66
	<i>Llama-3.1-8B</i>	59.30	59.24	59.21	59.30	44.29	43.96	43.86	44.29
	<i>Llama3.1-Aloe-Beta-8B</i>	65.50	65.40	65.60	65.50	54.01	53.64	53.70	54.01
	<i>Llama3-Med42-8B</i>	67.92	67.88	68.04	67.92	54.55	54.14	54.21	54.55

Table 5. Performance on MedMCQA under HERALD partial securing.[†]

changes. Gains were especially pronounced when answer options were left in plaintext, aligning with clinical practice and reducing vocabulary drift. Fuzzy Hash and FPE gen-

erally offered the strongest trade-offs between security and learnability. The recovery was larger on MedMCQA than MedQA-USMLE, suggesting that reasoning-heavy datasets

Model	Accuracy (%)	F ₁ -score (%)	Precision (%)	Recall (%)
<i>Qwen-2.5-7B</i>	56.95	56.84	56.91	56.95
<i>Mistral-7B-Instruct-v0.3</i>	47.09	47.07	47.17	47.09
<i>Llama-3.1-8B</i>	53.26	53.30	53.49	53.26
<i>Llama-3.1-Aloe-Beta-8B</i>	<u>56.25</u>	<u>56.12</u>	<u>56.60</u>	<u>56.25</u>
<i>Llama-3-Med42-8B</i>	51.99	50.20	54.23	51.99

Table 6. Baseline performance on MedQA-USMLE with plaintext data.[†]

are more vulnerable to obfuscation. Appendix A.4.1 presents attention maps across multiple textual variants, highlighting where the model attends.

Allowing MCQ answer options to remain unsecured (OU) is a pragmatic, low-risk choice that keeps the evaluation faithful to real clinical use while preserving utility. In standard medical MCQ corpora, the options are generic medical information; HERALD’s threat model therefore focuses protection on the stem, where context and identifiers may appear while leaving the public label space in clear text. This design mirrors deployment reality: clinicians see choices in plaintext, but any patient-linked content is secured before it ever reaches a model. Methodologically, OU avoids unnecessary vocabulary inflation and distributional drift that would arise from securing label tokens, yielding cleaner comparisons to plaintext baselines and stronger performance—consistent with our findings. Our controlled variants (FS+OU/PS+OU vs. FS+OFS/PS+OPS) make this explicit; across MedMCQA and MedQA-USMLE, leaving options unsecured consistently yields higher utility—for example, PS+OU outperforms PS+OPS for Qwen-2.5-7B on MedMCQA ($\approx 64.94\%$ vs. 55.83%) and MedQA-USMLE ($\approx 48.86\%$ vs. 39.91%), with similar trends across models and ciphers.

Effect of cryptographic methods. The choice of cryptographic primitive significantly shaped outcomes. Deterministic ciphers such as AES (ECB or SIV) and Blowfish preserved consistent mappings but induced steep drops under full security; their partial variants fared better, restoring performance into the mid-50s. Format-preserving encryption and similarity-preserving hashes (Fuzzy Hash, FPE) consistently outperformed classical hashes (MD5, SHA-1), which tended to erase too much distributional signal. The observed discrepancies across datasets likely stem from differences in reasoning depth, token distribution, and reliance on contextual semantics. Overall, HERALD demonstrates that carefully chosen cryptographic methods, applied selectively, can provide privacy guarantees while retaining strong downstream utility.

3.4.3. Training/Inference Wall-Clock

We present a wall-clock performance comparison between plaintext fine-tuning and HERALD (Table 9). Training throughput is computed as post-template input tokens per epoch divided by epoch wall-time. Inference latency is measured with single-example decoding over $n=128$ prompts (max 128 new tokens), reporting mean and p50/p90/p95, plus in-

puts/sec and input-tokens/sec. HERALD increases epoch time by $\sim 30\%$ due to longer tokenized sequences from delimiters and ciphertext, yet *tokens/sec* rises ($+14\%$), indicating better amortization of fixed overheads. At inference, HERALD raises median latency by $\sim 15\%$ and reduces inputs/sec (-10%), while input-tokens/sec nearly doubles ($\sim 1.8\times$), reflecting longer inputs.

3.4.4. Adversarial Evaluation and Leakage Analysis.

We stress-test the system under adversarial conditions: since training data are transformed, any memorization targets ciphertext rather than raw medical records (Appendix C.1; jailbreak and prompt injection in Appendix C.2). We also examine potential structural leakage from deterministic schemes using semantic-similarity and token-recovery probes (Appendix C.4, C.5), and quantify memory use, efficiency, and resource costs of the cryptographic pipeline (Appendix A.8).

4. Discussion

4.1. Benefits

HERALD offers several advantages for privacy-preserving NLP in the medical domain and beyond.

Strong Privacy for Key Tokens. By encrypting identifiers and other sensitive spans with standard cryptographic primitives, HERALD renders these tokens computationally indecipherable to any party lacking the key. This satisfies stringent privacy requirements (e.g., HIPAA in healthcare) by preventing disclosure of personal data. Even if an adversary exfiltrates model weights or activations, the sensitive content appears only as ciphertext, limiting direct leakage to plaintext values.

High Utility via Selective Encryption. In contrast to fully encrypted computation, which often yields substantial accuracy degradation [14], HERALD preserves a large fraction of task performance by encrypting only sensitive tokens while keeping non-sensitive context in clear text. Retaining context preserves grammatical and discourse relations that support inference, while ciphertext provides token identities that the model can learn as an auxiliary lexicon. Inference is performed by conditioning on both preserved plaintext and stable ciphertext types. Empirically, we observe that HERALD closes much of the gap between plaintext training and an all-encrypted baseline: when encrypting every token reduces accuracy toward chance, selective encryption recovers a substantial portion of the original performance (see Section 3.4). These findings support a favorable privacy–utility trade-off.

Model-Agnostic and Efficient. HERALD requires no architectural changes or specialized hardware. Any transformer-based language model (or other token-based model) can be used after simple data preprocessing. Fine-tuning uses standard optimization, and inference cost is near plaintext models; the only added major overhead is the HERALD preprocessing step, which introduces additional latency and compute.

Cryptographic Methods	Model	FS + OU				FS + OFS			
		Accuracy	F ₁ -score	Precision	Recall	Accuracy	F ₁ -score	Precision	Recall
AES (<i>SIV</i>)	<i>Qwen-2.5-7B</i>	31.74	31.48	32.07	31.74	24.98	14.83	13.31	24.98
	<i>Mistral-7B-Instruct-v0.3</i>	30.82	30.09	31.75	30.82	24.78	20.26	22.57	24.78
	<i>Llama-3.1-8B</i>	28.13	28.06	28.17	28.13	24.74	24.61	24.70	24.74
	<i>Llama3.1-Aloe-Beta-8B</i>	34.71	34.69	35.03	34.71	25.37	16.76	13.61	25.37
	<i>Llama3-Med42-8B</i>	34.08	33.70	33.99	34.08	25.59	15.49	32.12	25.59
Blowfish (<i>ECB</i>)	<i>Qwen-2.5-7B</i>	31.82	31.84	32.32	31.82	26.63	18.80	20.46	26.63
	<i>Mistral-7B-Instruct-v0.3</i>	29.34	28.24	29.76	29.34	24.19	20.24	21.34	24.19
	<i>Llama-3.1-8B</i>	28.49	28.44	28.60	28.49	24.67	24.49	24.62	24.67
	<i>Llama3.1-Aloe-Beta-8B</i>	34.64	34.56	34.78	34.64	<u>25.61</u>	17.33	13.77	<u>25.61</u>
	<i>Llama3-Med42-8B</i>	34.48	34.33	34.64	34.48	0.00	0.00	0.00	0.001
FPE	<i>Qwen-2.5-7B</i>	33.72	33.51	33.91	33.72	25.69	18.70	26.25	25.69
	<i>Mistral-7B-Instruct-v0.3</i>	30.11	29.55	30.73	30.11	26.16	22.33	32.60	26.16
	<i>Llama-3.1-8B</i>	29.62	29.63	29.74	29.62	24.67	24.62	24.69	24.67
	<i>Llama3.1-Aloe-Beta-8B</i>	34.64	34.65	35.14	34.64	25.84	20.38	25.90	25.84
	<i>Llama3-Med42-8B</i>	<u>34.50</u>	<u>34.45</u>	<u>34.84</u>	<u>34.50</u>	26.71	19.98	<u>29.43</u>	26.71
Fuzzy Hash	<i>Qwen-2.5-7B</i>	32.53	32.44	32.52	32.53	27.40	21.92	26.82	27.40
	<i>Mistral-7B-Instruct-v0.3</i>	29.62	28.98	30.17	29.62	24.59	22.35	23.36	24.59
	<i>Llama-3.1-8B</i>	29.76	29.71	29.86	29.76	24.98	24.89	24.89	24.98
	<i>Llama3.1-Aloe-Beta-8B</i>	34.86	34.71	35.06	34.86	<u>26.71</u>	23.56	26.96	<u>26.71</u>
	<i>Llama3-Med42-8B</i>	28.65	24.13	29.87	28.65	25.45	20.66	24.90	25.45
MD5	<i>Qwen-2.5-7B</i>	30.97	30.83	31.85	30.97	24.21	16.04	33.95	24.21
	<i>Mistral-7B-Instruct-v0.3</i>	30.73	30.35	31.10	30.73	25.65	23.47	24.12	25.65
	<i>Llama-3.1-8B</i>	27.87	27.83	27.96	27.87	<u>24.86</u>	24.80	24.96	24.86
	<i>Llama3.1-Aloe-Beta-8B</i>	34.81	34.85	35.25	34.81	25.73	16.90	19.13	25.73
	<i>Llama3-Med42-8B</i>	<u>32.79</u>	<u>32.70</u>	<u>33.00</u>	<u>32.79</u>	24.74	15.37	13.45	24.74
SHA-1	<i>Qwen-2.5-7B</i>	32.36	32.27	32.81	32.36	23.68	15.68	18.01	23.68
	<i>Mistral-7B-Instruct-v0.3</i>	30.10	29.57	31.42	30.10	23.37	20.52	26.78	23.37
	<i>Llama-3.1-8B</i>	28.54	28.50	28.55	28.54	<u>24.15</u>	24.13	24.24	<u>24.15</u>
	<i>Llama3.1-Aloe-Beta-8B</i>	35.41	35.45	35.81	35.41	26.20	18.15	21.79	26.20
	<i>Llama3-Med42-8B</i>	<u>33.80</u>	<u>33.66</u>	<u>33.85</u>	<u>33.80</u>	0.00	0.00	0.001	0.00

Table 7. Performance on MedQA-USMLE with fully secured tokens.[†]

Cryptographic Methods	Model	HERALD (PS + OU)				HERALD (PS + OPS)			
		Accuracy	F ₁ -score	Precision	Recall	Accuracy	F ₁ -score	Precision	Recall
AES (<i>SIV</i>)	<i>Qwen-2.5-7B</i>	47.06	47.07	47.12	47.06	36.45	36.22	36.43	36.45
	<i>Mistral-7B-Instruct-v0.3</i>	41.62	41.52	41.57	41.62	32.97	32.47	33.12	32.97
	<i>Llama-3.1-8B</i>	40.38	40.40	40.47	40.38	28.95	28.94	28.98	28.95
	<i>Llama-3.1-Aloe-Beta-8B</i>	48.00	48.00	48.23	48.00	35.80	35.70	<u>35.82</u>	<u>35.80</u>
	<i>Llama-3-Med42-8B</i>	44.35	44.25	44.33	44.35	32.94	32.63	32.76	32.94
Blowfish (<i>ECB</i>)	<i>Qwen-2.5-7B</i>	46.58	46.52	46.62	46.58	35.04	34.96	35.05	35.04
	<i>Mistral-7B-Instruct-v0.3</i>	42.42	42.41	42.70	42.42	33.54	33.19	33.26	33.54
	<i>Llama-3.1-8B</i>	41.48	41.48	41.61	41.48	30.40	30.35	30.49	30.40
	<i>Llama-3.1-Aloe-Beta-8B</i>	<u>47.92</u>	<u>47.95</u>	48.08	<u>47.92</u>	36.45	36.47	36.86	36.45
	<i>Llama-3-Med42-8B</i>	48.95	48.89	49.00	48.95	34.21	32.39	33.16	34.21
FPE	<i>Qwen-2.5-7B</i>	47.76	47.76	47.89	47.76	37.16	36.96	36.99	37.16
	<i>Mistral-7B-Instruct-v0.3</i>	43.44	43.29	43.55	43.44	37.31	37.13	37.67	37.31
	<i>Llama-3.1-8B</i>	43.36	43.33	43.44	43.36	32.44	32.43	32.58	32.44
	<i>Llama-3.1-Aloe-Beta-8B</i>	49.73	49.76	50.11	49.73	39.28	39.22	39.53	39.28
	<i>Llama-3-Med42-8B</i>	48.94	48.85	48.99	48.94	37.00	36.81	36.92	37.00
Fuzzy Hash	<i>Qwen-2.5-7B</i>	48.86	48.70	48.77	48.86	39.91	39.85	39.91	39.91
	<i>Mistral-7B-Instruct-v0.3</i>	42.03	42.06	42.21	42.03	35.90	35.59	35.62	35.90
	<i>Llama-3.1-8B</i>	42.58	42.56	42.70	42.58	32.29	32.24	32.31	32.29
	<i>Llama-3.1-Aloe-Beta-8B</i>	48.63	48.60	48.87	48.63	38.57	38.49	38.55	38.57
	<i>Llama-3-Med42-8B</i>	48.42	48.25	48.44	48.42	<u>38.60</u>	<u>38.50</u>	<u>38.57</u>	<u>38.60</u>
MD5	<i>Qwen-2.5-7B</i>	46.03	46.07	46.13	46.03	35.51	35.20	35.47	35.51
	<i>Mistral-7B-Instruct-v0.3</i>	39.89	39.94	40.37	39.89	33.99	33.25	33.73	33.99
	<i>Llama-3.1-8B</i>	40.85	40.82	41.08	40.85	28.54	28.55	28.64	28.54
	<i>Llama-3.1-Aloe-Beta-8B</i>	46.74	46.67	46.94	46.74	36.32	36.14	36.35	36.32
	<i>Llama-3-Med42-8B</i>	48.17	48.18	48.56	48.17	<u>36.30</u>	36.19	<u>36.35</u>	<u>36.30</u>
SHA-1	<i>Qwen-2.5-7B</i>	44.57	44.53	44.73	44.57	35.67	35.44	35.72	35.67
	<i>Mistral-7B-Instruct-v0.3</i>	42.60	42.58	43.09	42.60	32.68	32.15	32.53	32.68
	<i>Llama-3.1-8B</i>	40.08	40.08	40.15	40.08	28.43	28.42	28.59	28.43
	<i>Llama-3.1-Aloe-Beta-8B</i>	46.93	46.90	46.99	46.93	34.65	34.34	34.79	34.65
	<i>Llama-3-Med42-8B</i>	<u>46.10</u>	<u>45.83</u>	<u>45.99</u>	<u>46.10</u>	<u>35.20</u>	<u>34.63</u>	<u>35.25</u>	<u>35.20</u>

Table 8. Performance on MedQA-USMLE under HERALD partial securing.[†]

This stands in contrast to homomorphic-encryption-based inference, which is typically orders of magnitude slower [15]. As a result, the method scales readily to large models and datasets.

Flexible and Adaptable. The framework is highly configurable. Practitioners can specify which token classes to encrypt, extending sensitive-span identification with domain dictionaries or stronger entity recognizers as needed. They

Variant	Ep.	Time/ep (s)	Train tok/s	Train wall (s)	Inf p50 (ms)	Inf p90 (ms)	Inputs/s	Input toks/s
Plaintext	2	3845.0	333.6	7690.1	168.9	205.8	5.47	432.2
HERALD (PS+OU)	2	5009.2	380.5	10018.3	194.1	251.0	4.93	781.0
HERALD / Plaintext		×1.30	×1.14	×1.30	×1.15	×1.22	0.90×	×1.81

Table 9. **Wall-clock performance for plaintext vs. HERALD (PS+OU).** Columns: **Variant** training setup; **Ep.** epochs; **Time/ep (s)** mean seconds per epoch; **Train tok/s** throughput (post-template); **Train wall (s)** end-to-end time incl. eval and checkpoints; **Inf p50/p90 (ms)** median and 90th-percentile per-example latency; **Inputs/s** per-example generations per second; **Input toks/s** prompt tokens per second. Inference uses $n=128$ prompts. The last row reports HERALD/plaintext ratios.

may also choose among encryption mechanisms, for example, format-preserving encryption to retain length information. Marker tokens can optionally encode type information (e.g., using distinct markers for names versus dates), enabling the model to condition on entity type at inference; exploring such typed markers is left for future work.

Black-Box Compatibility. Because encryption occurs outside the model, HERALD supports settings where the model is exposed only as an API. Queries can be transformed client-side, sent to a third-party service, and any returned ciphertext spans can be post-processed (e.g., decrypted client-side when appropriate). Consequently, organizations can leverage powerful external LLMs without revealing raw sensitive content to the provider, strengthening trust and compliance.

Mitigation of Internal Memorization. Since the model never observes plaintext sensitive tokens, the risk of memorizing and reproducing them is greatly reduced. Even if overfitting occurs on encrypted spans, any subsequent regurgitation would consist of ciphertext, which is not directly decipherable (and is computationally infeasible to invert) without the key. This behavior complements orthogonal defenses such as differentially private fine-tuning; one can apply HERALD and DP jointly for additional protection, though in our use case we found DP to be unnecessary in practice.

In summary, HERALD provides a practical mechanism for **privacy-preserving language model fine-tuning** in domains that intermix sensitive and non-sensitive text. Operating at the token level with deterministic encryption and explicit markers enables training high-capacity transformer models on transformed data with minimal utility loss while shielding sensitive content from external adversaries and inadvertent model disclosure. We view this as an “**atomic-level obfuscation**” strategy: transform only what is necessary to protect privacy and leave the rest intact, thereby delivering strong protections against external breaches and internal leakage without sacrificing the benefits of large-scale language modeling.

4.2. Clinical Deployment Implications

HERALD is most relevant in settings where clinical text must cross institutional or vendor trust boundaries, yet full plaintext exposure is unacceptable and fully encrypted computation is impractical to operationalize. The key observation is

that clinical notes are not uniformly sensitive: the highest-risk content is concentrated in PHI and clinically identifying spans, whereas much of the remaining syntax, discourse, and document structure is needed for model utility. HERALD is therefore well matched to clinical deployment because it protects the spans that create disclosure risk while preserving the linguistic scaffold that lets standard LLMs remain useful. The following scenarios illustrate where HERALD could be most relevant in practice.

Scenario 1: Multi-center rare-disease model development. Consider a consortium of tertiary hospitals building a triage model for a rare autoimmune condition. No single site has enough cases to train a robust model, but raw report sharing is not permissible. With HERALD, each hospital keeps the original note inside its own EHR and exports only a transformed version in which PHI and disease-bearing spans are deterministically encrypted under a shared study policy. The central model never receives plaintext, yet it still learns from stable encrypted token identities embedded within readable context such as temporality, negation, severity, and assessment structure. This is precisely where HERALD is preferable to both naive de-identification and full-token encryption: the former leaves semantic leakage, while the latter removes the scaffold required for cross-site generalization. Clinically, the documentation workflow is unchanged, but collaborative model development becomes feasible.

Scenario 2: Vendor-hosted documentation and coding support. A hospital uses a cloud service to draft discharge summaries and suggest diagnosis or billing codes, but institutional policy prohibits sending patient-linked narratives in plaintext. HERALD is inserted as a client-side preprocessing layer at the workstation or gateway. Before transmission, only sensitive spans are transformed; section headers, ordinary syntax, and non-sensitive clinical context remain readable. The remote model is trained and served on transformed inputs, and the authorized client restores protected content locally in the final output. This makes the pipeline clinically reasonable because it preserves existing practice: clinicians still review ordinary notes, the vendor stack requires no architectural change, and the trust boundary is moved to the point of export. The cloud can support documentation without possessing the raw identifiers or disease strings that generated the note.

Scenario 3: Cross-institution surveillance under disclosure constraints. A regional network wants to monitor complications, treatment drift, or respiratory surge patterns across hospitals, but direct sharing of case-level reports or explicit disease statistics is difficult. HERALD allows each site to contribute transformed notes for centralized risk modeling while keeping protected spans in ciphertext throughout storage, transmission, and analysis. Because repeated sensitive entities map to stable token identities, the model can still detect recurrence, co-occurrence, and temporal change; because non-sensitive syntax remains intact, it can still use chronology, uncertainty, negation, and severity cues. In this setting, HERALD does not merely hide text. It enables a realistic governance model in which institutions retain local control of plaintext while still participating in shared clinical intelligence.

Clinical takeaway. HERALD is best viewed as a boundary technology for real clinical NLP systems. It is particularly important in scenarios where clinical text must be shared beyond the originating institution, where utility depends on preserving contextual meaning, and where deployment must remain compatible with standard tokenizers, existing LLMs, and routine computational infrastructure. In such settings, selective token-level protection provides a practical approach to privacy, aligning with how clinical text is generated, shared, and operationalized in real-world clinical pipelines.

4.3. Limitations

While HERALD enforces that protected spans are never sent to, or emitted by, the model in plaintext, it provides computational rather than information-theoretic privacy. Because surrounding context remains visible, an adversary could infer the meaning of stable ciphertext tokens via co-occurrence patterns or frequency analysis; in effect, the method raises, but does not eliminate, the cost of statistical inference from plaintext context. In addition, several primitives deliberately introduce determinism to preserve learnability, which leaks structure: ECB-style mappings create consistent ciphertext vocabularies that models (and attackers) can recognize, and 64-bit block ciphers increase susceptibility to repetition and birthday-bound effects on large corpora. Even format-preserving encryption leaks equality by construction, and similarity-preserving hashes, while useful for utility, are unkeyed obfuscations, leaving them vulnerable to guess-and-check under strong auxiliary knowledge. The threat model also excludes key compromise; if the secret key is exposed, encryption confers no protection, practical key management (including rotation and multi-key deployments) is a solution for it. Operationally, securing tokens inflates sequence length and increases memory footprint and preprocessing time, introducing latency before training or inference even though the forward pass itself remains close to plaintext

cost. Post-processing poses an additional hygiene risk: decrypted renderings of model outputs may leak via logs if not handled carefully. Finally, the pipeline’s linguistic normalizations and targeted lemmatization, while aiding stability, can induce subtle semantic drift; marker-design choices also balance learnability against potential type leakage. Empirically, we observe that reasoning-heavy tasks remain more sensitive to any obfuscation than knowledge-recall or short-context settings, which limits absolute recovery under partial security.

5. Conclusion

To balance efficiency and security, this study proposes HERALD, a token-level cryptographic redaction framework. HERALD protects sensitive spans while retaining non-sensitive context, enabling fine-tuning and inference on transformed clinical text without modifying model architectures. Across experiments on multiple tasks, fully securing all tokens imposed steep utility costs, whereas selective partial securing (using HERALD) recovered substantial accuracy without specialized hardware or model modifications. Keeping answer options in plaintext aligned with clinical workflows and consistently improved utility. Among primitives, fuzzy hashing and format-preserving encryption offered the best privacy–utility trade-offs. HERALD is model-agnostic, works with black-box APIs via client-side transformation, and complements defenses such as differentially private fine-tuning. Overall, our paper shows that selective securing can provide strong, practical privacy for medical NLP while preserving performance, and we expect HERALD to spur further research on principled token-level security.

Data Availability

All data analyzed in this study were obtained from publicly available benchmark datasets. The Medical Abstracts Text Classification (Med-TC) dataset is available at https://huggingface.co/datasets/TimSchopf/medical_abstracts. The MedMCQA dataset is available at <https://huggingface.co/datasets/openlifescienceai/medmcqa>. The MedQA-USMLE dataset is available at <https://huggingface.co/datasets/GBaker/MedQA-USMLE-4-options-hf>. Dataset descriptions, task-specific usage, and train/validation/test split details are provided in Section 3.2.

References

- [1] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutierrez, T. F. Tan, and D. S. W. Ting, “Large language models in medicine,” *Nature medicine*, vol. 29, no. 8, pp. 1930–1940, 2023. 1
- [2] H. Xiao, F. Zhou, X. Liu, T. Liu, Z. Li, X. Liu, and X. Huang, “A comprehensive survey of large language models and mul-

- timodal large language models in medicine,” *arXiv preprint arXiv:2405.08603*, 2024.
- [3] Z. A. Nazi and W. Peng, “Large language models in healthcare and medical domain: A review,” in *Informatics*, vol. 11, p. 57, MDPI, 2024.
 - [4] F. Gaber, M. Shaik, F. Allega, A. J. Bilecz, F. Busch, K. Goon, V. Franke, and A. Akalin, “Evaluating large language model workflows in clinical decision support for triage and referral and diagnosis,” *npj Digital Medicine*, vol. 8, no. 1, p. 263, 2025.
 - [5] J. Huang, D. M. Yang, R. Rong, K. Nezafati, C. Treager, Z. Chi, S. Wang, X. Cheng, Y. Guo, L. J. Klesse, *et al.*, “A critical assessment of using chatgpt for extracting structured data from clinical notes,” *NPJ digital medicine*, vol. 7, no. 1, p. 106, 2024. 1, 8
 - [6] OpenAI, “ChatGPT (July 2025 version).” [Large language model], 2025. Accessed via chat.openai.com. 1
 - [7] G. Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican, *et al.*, “Gemini: a family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2023. 1
 - [8] J. Jonnagaddala and Z. S.-Y. Wong, “Privacy preserving strategies for electronic health records in the era of large language models,” *npj Digital Medicine*, vol. 8, no. 1, p. 34, 2025. 1
 - [9] R. Staab, M. Vero, M. Balunović, and M. Vechev, “Beyond memorization: Violating privacy via inference with large language models,” *arXiv preprint arXiv:2310.07298*, 2023. 1
 - [10] S. Ishihara, “Training data extraction from pre-trained language models: A survey,” *arXiv preprint arXiv:2305.16157*, 2023.
 - [11] B. Peng, K. Chen, Q. Niu, Z. Bi, M. Liu, P. Feng, T. Wang, L. K. Yan, Y. Wen, Y. Zhang, *et al.*, “Jailbreaking and mitigation of vulnerabilities in large language models,” *arXiv preprint arXiv:2410.15236*, 2024. 1
 - [12] S. A. Tovino, “Artificial intelligence and the hipaa privacy rule: A primer,” *Houston Journal of Health Law & Policy*, vol. 24, no. 1, pp. 77–126, 2025. 2
 - [13] A. K. Y. Yanamala, S. Suryadevara, and V. D. R. Kalli, “Balancing innovation and privacy: The intersection of data protection and artificial intelligence,” *International Journal of Machine Learning Research in Cybersecurity and Artificial Intelligence*, vol. 15, no. 1, pp. 1–43, 2024. 2
 - [14] G. Zhao and E. Song, “Privacy-preserving large language models: Mechanisms, applications, and future directions,” *arXiv preprint arXiv:2412.06113*, 2024. 2, 14
 - [15] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, “Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy,” in *International conference on machine learning*, pp. 201–210, PMLR, 2016. 15
 - [16] L. de Castro, D. Escudero, A. Agrawal, A. Polychroniadou, and M. Veloso, “EncryptedLLM: Privacy-preserving large language model inference via GPU-accelerated fully homomorphic encryption,” in *Forty-second International Conference on Machine Learning*, 2025.
 - [17] D. Rho, T. Kim, M. Park, J. W. Kim, H. Chae, E. K. Ryu, and J. H. Cheon, “Encryption-friendly llm architecture,” *arXiv preprint arXiv:2410.02486*, 2024. 2
 - [18] J. Majmudar, C. Dupuy, C. Peris, S. Smaili, R. Gupta, and R. Zemel, “Differentially private decoding in large language models,” *arXiv preprint arXiv:2205.13621*, 2022. 2
 - [19] X. Li, B. Zhao, G. Yang, T. Xiang, J. Weng, and R. H. Deng, “A survey of secure computation using trusted execution environments,” *arXiv preprint arXiv:2302.12150*, 2023.
 - [20] F. Mo, Z. Tarkhani, and H. Haddadi, “Machine learning with confidential computing: A systematization of knowledge,” *ACM computing surveys*, vol. 56, no. 11, pp. 1–40, 2024. 2
 - [21] L. Peng, G. Luo, S. Zhou, J. Chen, Z. Xu, J. Sun, and R. Zhang, “An in-depth evaluation of federated learning on biomedical natural language processing for information extraction,” *NPJ Digital Medicine*, vol. 7, no. 1, p. 127, 2024. 2
 - [22] Z. Kan, L. Qiao, H. Yu, L. Peng, Y. Gao, and D. Li, “Protecting user privacy in remote conversational systems: A privacy-preserving framework based on text sanitization,” *arXiv preprint arXiv:2306.08223*, 2023. 2
 - [23] A. R. Chowdhury, D. Glukhov, D. Anshuman, P. Chalasani, N. Papernot, S. Jha, and M. Bellare, “Preempt: Sanitizing sensitive prompts for llms,” 2025.
 - [24] A. Mishra, M. Li, and S. Deo, “Sentinellms: encrypted input adaptation and fine-tuning of language models for private and secure inference,” in *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI’24/IAAI’24/EAAI’24, AAAI Press, 2024. 2, 5, 6, 8
 - [25] S. Lin, W. Hua, Z. Wang, M. Jin, L. Fan, and Y. Zhang, “EmojiPrompt: Generative prompt obfuscation for privacy-preserving communication with cloud-based LLMs,” in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (L. Chiruzzo, A. Ritter, and L. Wang, eds.), (Albuquerque, New Mexico), pp. 12342–12361, Association for Computational Linguistics, Apr. 2025. 2
 - [26] B. Norgeot, K. Muenzen, T. A. Peterson, X. Fan, B. S. Glicksberg, G. Schenk, E. Rutenber, B. Oskotsky, M. Sirota, J. Yazdany, *et al.*, “Protected health information filter (philter): accurately and securely de-identifying free-text clinical notes,” *NPJ digital medicine*, vol. 3, no. 1, p. 57, 2020.
 - [27] A. Noonian, D. Thakral, P. Mathur, F. Sheth, H. Shaikh, and A. K. Gupta, “A discrete mathematical model and cryptography for secure medical image analysis : Encrypted chest x-ray classification,” *Journal of Discrete Mathematical Sciences and Cryptography*, vol. 28, no. 5-A, p. 1473–1486, 2025. 2
 - [28] Z. Yang, H. Huangfu, M. Ran, Z. Wang, H. Yu, M. Sun, and Y. Zhang, “A novel privacy-enhancing framework for low-dose ct denoising,” *IEEE Transactions on Artificial Intelligence*, 2025. 2
 - [29] D. Khyani, B. Siddhartha, N. Niveditha, and B. Divya, “An interpretation of lemmatization and stemming in natural language processing,” *Journal of University of Shanghai for Science and Technology*, vol. 22, no. 10, pp. 350–357, 2021. 4

- [30] G. A. Miller, "Wordnet: a lexical database for english," *Communications of the ACM*, vol. 38, no. 11, pp. 39–41, 1995. 5
- [31] A. Paul, D. Shaji, L. Han, W. Del-Pinto, and G. Nenadic, "Deidclinic: A multi-layered framework for de-identification of clinical free-text data," *arXiv preprint arXiv:2410.01648*, 2024. 5, 6
- [32] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in python," 2020. 5
- [33] J. Buban, H. Zhang, C. Angione, H. Yang, A. Farhan, S. Sultanov, M. Du, X. Ma, Z. Wang, Y. Zhao, *et al.*, "Encrypted large model inference: The equivariant encryption paradigm," *arXiv preprint arXiv:2502.01013*, 2025. 6
- [34] C. Qu, W. Kong, L. Yang, M. Zhang, M. Bendersky, and M. Najork, "Natural language understanding with privacy-preserving bert," in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, pp. 1488–1497, 2021. 6
- [35] G. Singh, "A study of encryption algorithms (rsa, des, 3des and aes) for information security," *International Journal of Computer Applications*, vol. 67, no. 19, 2013. 6
- [36] X. Li, F. Tramer, P. Liang, and T. Hashimoto, "Large language models can be strong differentially private learners," *arXiv preprint arXiv:2110.05679*, 2021. 7
- [37] I. C. Wiest, D. Ferber, J. Zhu, M. van Treeck, S. K. Meyer, R. Juglan, Z. I. Carrero, D. Paech, J. Kleesiek, M. P. Ebert, *et al.*, "Privacy-preserving large language models for structured medical information retrieval," *NPJ Digital Medicine*, vol. 7, no. 1, p. 257, 2024. 8
- [38] K. Singhal, S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, N. Scales, A. Tanwani, H. Cole-Lewis, S. Pfohl, *et al.*, "Large language models encode clinical knowledge," *Nature*, vol. 620, no. 7972, pp. 172–180, 2023. 8
- [39] H. Nori, N. King, S. M. McKinney, D. Carignan, and E. Horvitz, "Capabilities of gpt-4 on medical challenge problems," *arXiv preprint arXiv:2303.13375*, 2023. 8
- [40] M. Bekbolatova, J. Mayer, C. W. Ong, and M. Toma, "Transformative potential of ai in healthcare: definitions, applications, and navigating the ethical landscape and public perspectives," in *Healthcare*, vol. 12, p. 125, MDPI, 2024. 8
- [41] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019. 8
- [42] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019. 8
- [43] P. He, J. Gao, and W. Chen, "Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing," *arXiv preprint arXiv:2111.09543*, 2021. 8
- [44] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," 2019. 8
- [45] J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J. Kang, "Biobert: a pre-trained biomedical language representation model for biomedical text mining," *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, 2020. 8
- [46] G. Wang, X. Liu, Z. Ying, G. Yang, Z. Chen, Z. Liu, M. Zhang, H. Yan, Y. Lu, Y. Gao, *et al.*, "Optimized glycemic control of type 2 diabetes with reinforcement learning: a proof-of-concept trial," *Nature Medicine*, vol. 29, no. 10, pp. 2633–2642, 2023. 8
- [47] R. Luo, L. Sun, Y. Xia, T. Qin, S. Zhang, H. Poon, and T.-Y. Liu, "BioGPT: generative pre-trained transformer for biomedical text generation and mining," *Briefings in Bioinformatics*, vol. 23, 09 2022. bbac409. 8
- [48] Qwen, :, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, "Qwen2.5 technical report," 2025. 8
- [49] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," 2023. 8
- [50] Meta AI, "Introducing llama 3.1: Our most capable models to date." <https://ai.meta.com/blog/meta-llama-3-1/>, July 2024. Accessed: 2025-08-10. 8
- [51] D. Garcia-Gasulla, J. Bayarri-Planas, A. K. Gururajan, E. Lopez-Cuena, A. Tormos, D. Hincos, P. Bernabeu-Perez, A. Arias-Duart, P. A. Martin-Torres, M. Gonzalez-Mallo, *et al.*, "The aloe family recipe for open and specialized healthcare llms," *arXiv preprint arXiv:2505.04388*, 2025. 8
- [52] C. Christophe, P. K. Kanithi, T. Raha, S. Khan, and M. A. Pimentel, "Med42-v2: A suite of clinical llms," *arXiv preprint arXiv:2408.06142*, 2024. 8
- [53] T. Schopf, D. Braun, and F. Matthes, "Evaluating unsupervised text classification: Zero-shot and similarity-based approaches," in *Proceedings of the 2022 6th International Conference on Natural Language Processing and Information Retrieval, NLPPIR '22*, (New York, NY, USA), p. 6–15, Association for Computing Machinery, 2023. 8
- [54] A. Pal, L. K. Umapathi, and M. Sankarasubbu, "Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering," in *Proceedings of the Conference on Health, Inference, and Learning* (G. Flores, G. H. Chen, T. Pollard, J. C. Ho, and T. Naumann, eds.), vol. 174 of *Proceedings of Machine Learning Research*, pp. 248–260, PMLR, 07–08 Apr 2022. 8
- [55] D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, and P. Szolovits, "What disease does this patient have? a large-scale open domain question answering dataset from medical exams," *Applied Sciences*, vol. 11, no. 14, p. 6421, 2021. 9
- [56] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," *arXiv preprint arXiv:2009.03300*, 2020. 9

- [57] N. I. of Standards, T. (NIST), M. J. Dworkin, E. Barker, J. Nechvatal, J. Foti, L. E. Bassham, E. Roback, and J. D. Jr., "Advanced encryption standard (aes)," 2001-11-26 00:11:00 2001. [21](#)
- [58] N. Mouha, *Review of the advanced encryption standard*. US Department of Commerce, National Institute of Standards and Technology, 2021. [21](#)
- [59] S. Almuhammadi and I. Al-Hejri, "A comparative analysis of aes common modes of operation," in *2017 IEEE 30th Canadian Conference on Electrical and Computer Engineering (CCECE)*, pp. 1–4, 2017. [21](#)
- [60] B. D. Kim, V. A. Vasudevan, R. G. D'Oliveira, A. Cohen, T. Stahlbuhk, and M. Médard, "Cryptanalysis via machine learning based information theoretic metrics," *arXiv preprint arXiv:2501.15076*, 2025. [21](#)
- [61] P. Rogaway and T. Shrimpton, "A provable-security treatment of the key-wrap problem," in *Annual international conference on the theory and applications of cryptographic techniques*, pp. 373–390, Springer, 2006. [21](#)
- [62] S. Gueron and Y. Lindell, "Gcm-siv: full nonce misuse-resistant authenticated encryption at under one cycle per byte," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pp. 109–119, 2015. [21](#)
- [63] P. Rogaway and T. Shrimpton, "The siv mode of operation for deterministic authenticated-encryption (key wrap) and misuse-resistant nonce-based authenticated-encryption," *Aug*, vol. 20, p. 3, 2007. [21](#)
- [64] B. Schneier, "The blowfish encryption algorithm," *Dr Dobb's Journal-Software Tools for the Professional Programmer*, vol. 19, no. 4, pp. 38–43, 1994. [21](#)
- [65] B. Schneier, "Description of a new variable-length key, 64-bit block cipher (blowfish)," in *International workshop on fast software encryption*, pp. 191–204, Springer, 1993. [21](#)
- [66] M. Bellare, P. Rogaway, and T. Spies, "The ffx mode of operation for format-preserving encryption," *NIST submission*, vol. 20, no. 19, pp. 1–18, 2010. [21](#)
- [67] M. Dworkin, "Recommendation for block cipher modes of operation," *NIST special publication*, vol. 800, p. 38B, 2001. [21](#)
- [68] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proceedings of the thirty-fourth annual ACM symposium on Theory of computing*, pp. 380–388, 2002. [21](#), [22](#)
- [69] J. Kornblum, "Identifying almost identical files using context triggered piecewise hashing," *Digital investigation*, vol. 3, pp. 91–97, 2006. [22](#)
- [70] G. S. Manku, A. Jain, and A. Das Sarma, "Detecting near-duplicates for web crawling," in *Proceedings of the 16th international conference on World Wide Web*, pp. 141–150, 2007. [22](#)
- [71] X. Wang and H. Yu, "How to break md5 and other hash functions," in *Annual international conference on the theory and applications of cryptographic techniques*, pp. 19–35, Springer, 2005. [22](#)
- [72] R. Rivest, "The md5 message-digest algorithm," tech. rep., 1992. [22](#)
- [73] J. Black, M. Cochran, and T. Highland, "A study of the md5 attacks: Insights and improvements," in *International Workshop on Fast Software Encryption*, pp. 262–277, Springer, 2006. [22](#)
- [74] C. De Canniere and C. Rechberger, "Finding sha-1 characteristics: General results and applications," in *International conference on the theory and application of cryptology and information security*, pp. 1–20, Springer, 2006. [22](#)
- [75] M. Stevens, E. Bursztein, P. Karpman, A. Albertini, and Y. Markov, "The first collision for full sha-1," in *Annual international cryptology conference*, pp. 570–596, Springer, 2017. [22](#)
- [76] F. Pub, "Secure hash standard (shs)," *Fips pub*, vol. 180, no. 4, p. 2012, 2012. [22](#)

A. Supplementary Information

A.1. Encryption-Based Methods

A.1.1. Symmetric-Key Encryption

We use two secret-key block ciphers, **Advanced Encryption Standard (AES)** and **Blowfish**, to obtain reversible one-to-one mappings from plaintext tokens to ciphertext tokens under a shared key. Such consistent mappings are important for model learnability because identical plaintext tokens must consistently map to identical ciphertext tokens.

Advanced Encryption Standard (AES). AES (FIPS 197) is a substitution–permutation network operating on a 128-bit state [57, 58]. The number of rounds N_r is 10, 12, or 14 for 128-, 192-, or 256-bit keys, respectively. For a plaintext block P and key K , encryption produces $C = E_K(P)$, where E_K denotes the AES block-cipher encryption function; decryption $D_K(C)$ exactly inverts $E_K(P)$ over $\text{GF}(2^8)$. We employ two modes:

(i) **AES in Electronic Codebook (ECB) mode.** ECB encrypts each block independently,

$$C_i = E_K(P_i) \quad (3)$$

which is deterministic and therefore leaks block-level repetition [59]. Although ECB is semantically insecure for general-purpose encryption (e.g., the “ECB penguin” visual example²), within HERALD this determinism is intentional: it enforces a stable ciphertext vocabulary so that repeated sensitive terms (e.g., “aspirin”) map to the same secure token, effectively yielding a private but learnable lexicon. Prior work shows that ML models can detect ECB-induced patterns [60]; here we exploit that property to support learning while relying on secrecy of the 128-bit key to prevent disclosure of plaintext content.

(ii) **AES in Synthetic Initialization Vector (SIV) mode.** SIV is an *authenticated encryption with associated data* (AEAD) mode designed for high-security applications and resistance to nonce-misuse [61]. It computes a deterministic “synthetic IV” V via S2V (CMAC-based) under an authentication key K_1 and then encrypts using a counter mode under key K_2 :

$$V = \text{S2V}_{K_1}(\text{AD}_1, \dots, \text{AD}_m, P) \quad (4)$$

$$C = E_{K_2}(V, P) \quad (5)$$

If a nonce is reused, SIV does not catastrophically fail; it reveals only equality of (AD, P) while preserving confidentiality and integrity [62, 63]. In HERALD, AES (SIV) provides high-assurance deterministic encryption for critical tokens where pattern leakage must be minimized and authenticity guaranteed. We use a 256-bit total key split into two 128-bit keys for S2V and encryption.

Blowfish. Blowfish is a 16-round Feistel cipher operating on 64-bit blocks with variable key sizes from 32 to 448 bits [64, 65]. We use a 128-bit key. Its expensive key-schedule precomputes key-dependent S-boxes, making frequent re-keying slow but bulk encryption fast. As with AES (ECB), we use Blowfish in ECB mode to obtain a deterministic, learnable mapping:

$$C_i = E_K^{\text{Blowfish}}(P_i) \quad (6)$$

The 64-bit block size increases susceptibility to structural leakage and birthday-bound³ considerations on very large corpora, but the risk is reduced in our setting where we encrypt short, individual tokens.

Practical key sizes, ciphertext footprint, and deployment guidance are detailed in Appendix A.7.1.

A.1.2. Format-Preserving Schemes

We also consider transformations that preserve format to ease integration with NLP pipelines and tokenizers.

Format-Preserving Encryption (FPE). Conventional modes (e.g., ECB, CBC) often yield binary ciphertexts that do not resemble natural text, which can disrupt tokenization and distributional properties expected by language models. FPE defines a keyed permutation over a finite domain (*fixed length and alphabet*), producing ciphertexts that match the plaintext’s length and character set [66]. Determinism (no random IV) ensures that the same input under the same key yields the same output. This property is advantageous for NLP: ciphertext tokens retain word-like statistics and remain compatible with existing tokenizers. For example, a 5-letter token such as “fever” may map to a length-5 ciphertext like “4;cbP” under an alphabet of letters, digits, and punctuation. We instantiate FF1-style FPE built over AES with a 128-bit key. While FPE leaks repetitions (identical plaintexts map to identical ciphertexts), its security inherits from AES and is widely regarded as suitable for sensitive structured data [67]. Within HERALD, FPE is used for structured medical fields so that encrypted outputs remain “plaintext-like,” simplifying downstream processing and maintaining compatibility with existing workflows.

A.2. Obscuration Methods

A.2.1. Similarity Hashing

Similarity hashing, a family of locality-sensitive hashing (LSH) techniques, preserves relationships between inputs in the hash space [68]. Unlike encryption, these transformations are unkeyed and one-way: they intentionally give up *cryptographic reversibility* in order to retain semantic structure. Deterministic encryption (e.g., ECB) maps related tokens to unrelated ciphertext, erasing distributional

²[https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation#Electronic_codebook_\(ECB\)](https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation#Electronic_codebook_(ECB))

³[https://en.wikipedia.org/wiki/Blowfish_\(cipher\)#Weakness_and_successors](https://en.wikipedia.org/wiki/Blowfish_(cipher)#Weakness_and_successors)

cues that models rely on. In contrast, similarity hashes map nearby texts to nearby signatures, enabling models to exploit preserved structure. These methods are best viewed as strong obfuscation rather than encryption: they impede direct reading of the source text yet may permit inference by adversaries with side information or input-manipulation capability.

Fuzzy Hashing. We employ context-triggered piecewise hashing (CTPH) via *ssdeep*, which produces digests robust to small insertions, deletions, and edits [69]. The algorithm uses a rolling hash to select chunk boundaries, applies a conventional hash (e.g., FNV) to each chunk, and retains a few bits per chunk to build a compressed signature [69]. The resulting signature is longer than a conventional cryptographic hash and encodes partial information about the input’s content. When two plaintexts share large substrings, their fuzzy hashes share corresponding substrings; consequently, if two inputs differ only in minor details, their transformed representations overlap, allowing the model to treat them as similar. From a security perspective, fuzzy hashing is weaker than encryption because it is unkeyed and thus susceptible to guess-and-check attacks against candidate inputs. Yet the signature preserves only a small fingerprint of each chunk, making exact reconstruction difficult without auxiliary knowledge. We therefore treat fuzzy hashing as “partially secure”: it deters casual inspection and simple keyword matching, while offering no cryptographic guarantees under active, oracle-style attacks, which are out of scope in our black-box LLM setting. Its principal utility is that similarity in the original text translates into similarity in the obfuscated space, supporting generalization.

Soft Hashing. As a complementary similarity-preserving approach, we use *SimHash* [68] to produce fixed-length fingerprints. The algorithm is designed such that the Hamming distance between the fingerprints of two documents is a reliable approximation of the cosine similarity between their feature vectors [70]. The process involves tokenizing the input, weighting features (e.g., by TF-IDF), hashing each feature into a vector, and then combining these vectors to produce the final fingerprint. Compared with CTPH, SimHash offers a more explicitly vector-space view of semantic relatedness. From a privacy standpoint, it is also unkeyed and not collision-resistant; many distinct inputs intentionally converge when they share global similarity. This many-to-one mapping provides some anonymity because the original cannot be uniquely recovered from the fingerprint. By transforming sensitive tokens into SimHash fingerprints, HERALD allows the model to learn graded relationships among conceptually related medical terms in an obfuscated representation.

A.2.2. Cryptographic Hash Functions

Cryptographic hash functions provide deterministic, one-way mappings from arbitrary-length inputs to fixed-size digests. Their security rests on pre-image resistance (infeasible to find an input for a given output), second pre-image resistance (infeasible to find a second input that hashes to the same output as a given input), and collision resistance (infeasible to find any two distinct inputs that produce the same output) [71]. Inverting such hashes is intended to be computationally infeasible without exhaustive search.

MD5. Message Digest 5 (MD5) outputs a 128-bit digest [72]. It is cryptographically broken due to practical collision attacks [71], and thus unsuitable for collision-sensitive uses such as digital signatures [73]. Within HERALD, we restrict MD5 to high-throughput, one-way obfuscation where pre-image resistance is the primary concern. The probability that two random inputs collide is extremely small (2^{-128} per pair), and we observed no collisions when hashing individual tokens in our datasets. For short, high-entropy medical terms, finding a pre-image for a given digest remains computationally challenging despite MD5’s collision weaknesses. Its speed makes it useful for transforming large volumes of sensitive tokens.

SHA-1. SHA-1 is a 160-bit hash once standardized for federal use [74] but now deprecated after public collision and chosen-prefix collision attacks [75]. Like MD5, SHA-1 is deterministic and unkeyed, enabling rainbow-table style precomputation for common tokens⁴. In HERALD, its role mirrors MD5’s: a legacy, one-way obfuscation primitive that modestly increases digest length (160 vs. 128 bits) without the computational cost of modern designs such as SHA-256.

SHA-256. SHA-256, part of the SHA-2 family (FIPS 180-4), produces a 256-bit digest and is widely considered secure against practical attacks [76]. It processes 512-bit blocks over 64 rounds. In HERALD, SHA-256 is the default for irreversible token transformation when stronger security guarantees are required and performance is secondary, ensuring a high-assurance one-way mapping for sensitive tokens.

A.3. Ablation and External Evaluation Setup

Unless otherwise specified, all ablations and external evaluations use **Llama3-Med42-8B** fine-tuned on **MedMCQA** with LoRA. The text used for the tests is also taken from the MedMCQA dataset (although the exact text used may differ for each experiment). For discriminative tasks, we report accuracy (primary) and macro F_1 ; for generative tasks, we report BLEU, ROUGE-L, and BERTScore. All remaining training and inference settings follow Section 3.3.

⁴https://en.wikipedia.org/wiki/Rainbow_table

A.4. Interpretability Analysis

We investigate how models route attention when parts of the input are secured and how surrounding context supports recovery of task signal.

A.4.1. Attention over encrypted vs. plaintext regions

Setup. We visualize token-wise *incoming* attention mass for a representative MCQ item under five textual regimes: Base, FS+OU, FS+OFS, PS+OU (HERALD), and PS+OPS (HERALD). All panels use the **middle transformer layer (Layer 16 of 32)** and **average over all heads** to capture the dominant routing pattern while avoiding head cherry-picking. Concretely, we plot the mean attention *received* by each token from the rest of the sequence⁵. In HERALD, encrypted spans are delimited by sentinel tokens #^ and ^#; interior tokens are ciphertext produced by our token-level cryptographic transform. Under FS, all text is transformed (and, in FS+OFS, the answer options as well; for FS workflow see Section 2.1).

Qualitative findings. Figure 6 shows attention maps across regimes.

(A) Base. Mass concentrates on semantically informative stem tokens (*characteristic, fallopian, tube*) and on the correct option (*Watery discharge P/V*); instruction tokens attract minimal mass.

(B) FS+OU. With the stem fully secured, mass also shifts toward the plaintext options, peaking on the correct choice; though ciphertext within the stem also receives a comparable amount of direct mass. The model relies on the unchanged options to resolve the answer.

(C) FS+OFS. When both stem and options are fully secured, attention diffuses and comparatively re-weights toward prompt-template tokens (e.g., *Answer, option's, letter*) and positional anchors. This mirrors the large utility drop observed under FS+OFS: the model has few semantic footholds.

(D) HERALD (PS+OU). Attention exhibits strong peaks on encrypted spans, indicating that the model treats each protected block as a salient, cohesive unit despite being outside the natural-language lexicon and out-of-vocabulary (OOV). *Context compensation*: neighboring plaintext headwords (e.g., *characteristic*) and option tokens complement the cipher-block anchors, indicating the model uses preserved syntax/context together with the ciphertext units during reasoning.

(E) HERALD (PS+OPS). Securing both stem and options under HERALD preserves structured attention: mass splits between (i) ciphertext blocks corresponding to medically informative spans and (ii) plaintext shards and symbols that preserve option structure (e.g., “watery”). Compared with

FS+OFS, attention is less diffuse and re-centers on stable contextual anchors, consistent with the improved utility of HERALD.

Takeaways. (i) *Attention concentrates on secured blocks.* Across HERALD panels, ciphertext tokens attract the majority of mass, indicating that the model can use each protected block as a coherent anchor even though it is OOV. (ii) *Context still rescues semantics.* Despite hidden spans, attention also allocates to surrounding plaintext headwords and function words, supporting a “hide-in-plain-sight” design in which distributional cues carry residual task signal. (iii) *Over-encryption collapses routing.* Fully securing both stems and options (FS+OFS) yields diffuse, template-centric attention and degraded performance. Overall, HERALD concentrates attention on encrypted blocks while leveraging intact context, preserving workable reasoning paths without exposing private content.

A.5. Generative Answering Modes

Setup. We evaluate HERALD’s performance under generative MCQ answering, where models produce full-text option responses rather than option letters alone. This setting tests whether encrypted content preserves sufficient semantic coherence for free-form generation tasks. We compare five configurations: (1) **Baseline** (plaintext, no encryption), (2) **FS+OU** (fully secured stems with unsecured options), (3) **FS+OFS** (fully secured stems and options), (4) **HERALD (PS+OU)** (partially secured stems with unsecured options), and (5) **HERALD (PS+OPS)** (partially secured stems and options). For encrypted configurations, we test four cryptographic methods: AES (SIV), Blowfish (ECB), FPE, and AES (ECB). Models generate complete option text (e.g., “A. hypertension”) rather than single-letter predictions. Evaluation employs generative metrics: exact match (EM, strict full-option accuracy), BLEU, ROUGE-1/2/L, and BERTScore to assess semantic fidelity.

Findings. Table 10 presents comprehensive results across all configurations and cryptographic methods. The plaintext **Baseline** achieves the highest performance (EM: 73.8%, BLEU: 82.1%, ROUGE-1: 85.3%, BERTScore-F1: 88.9%), establishing the upper bound for utility without any privacy protection. HERALD’s **PS+OU** configurations demonstrate strong resilience, retaining approximately 87% of baseline exact match accuracy across encryption methods. FPE achieves the best performance with 64.7% EM (BLEU: 72.0%, BERTScore-F1: 82.8%), followed by AES (SIV) at 64.3% EM (BLEU: 71.6%, BERTScore-F1: 82.5%), AES (ECB) at 62.9% EM, and Blowfish (ECB) at 62.5% EM. When options are also partially secured (**PS+OPS**), performance degrades moderately: AES (SIV) maintains

⁵For readability, BOS/EOS tokens are omitted and attention is renormalized over the remaining sequence.

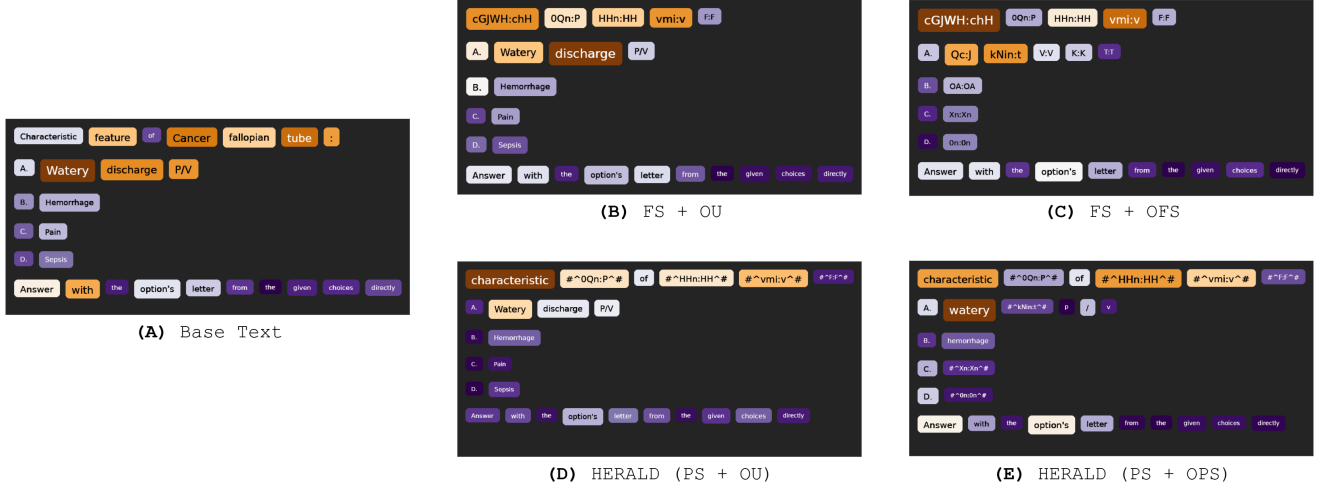


Figure 6. **Attention mapping under varying security regimes.** We visualize token-wise incoming attention at **Layer 16 (of 32), averaged over heads**. Panels: (A) Base, (B) **FS+OU** (fully secured stem; plaintext options), (C) **FS+OFS** (fully secured stem and options), (D) **HERALD PS+OU** (partially secured stem; plaintext options), and (E) **HERALD PS+OPS** (partially secured stem and options). In (D–E), spans wrapped by `#^...^#` are HERALD-encrypted blocks; in (B–C), all tokens (and options in C) are directly transformed. *Rendering*: darker brown, larger boxes denote higher attention to that token; smaller, darker purple boxes denote lower attention.

59.6% EM (BLEU: 66.1%, BERTScore-F1: 79.6%), representing a 4.7 percentage point reduction from PS+OU. Fully secured configurations exhibit substantial degradation. **FS+OU** achieves 45.8–49.3% EM across methods (FPE: 49.3%), while **FS+OFS** drops to 37.0–41.5% EM (FPE: 41.5%), with corresponding BLEU scores below 58% and BERTScore-F1 below 73%. Across all encrypted configurations, FPE demonstrates competitive performance with format-preserving properties, while AES (SIV) provides strong deterministic encryption, validating their complementary strengths for semantic preservation.

Implications. The generative evaluation reveals that HERALD’s selective encryption strategy maintains semantic coherence sufficient for free-form text generation, a more challenging task than option discrimination. The performance gap between PS+OU with FPE (64.7% EM) and plaintext baseline (73.8% EM)—only 9.1 percentage points—demonstrates that models can generate contextually appropriate responses even when sensitive content is obfuscated. This validates HERALD’s core hypothesis that preserving grammatical structure and non-sensitive context enables effective model operation under partial encryption. The steep decline in fully secured scenarios (FS+OFS: 41.5% EM for FPE, representing a 43.8% relative reduction from baseline) highlights the critical role of plaintext context in generative tasks, where models must synthesize coherent responses rather than simply discriminate among predefined options. BERTScore metrics reveal that even when exact matches fail, HERALD-secured models produce semantically

similar responses (BERTScore-F1: 82.8% for PS+OU with FPE vs. 88.9% baseline), indicating that generation errors manifest as synonym substitutions or paraphrasing rather than complete semantic drift. FPE’s format-preserving properties demonstrate slight advantages in most configurations, preserving token structure while maintaining strong security guarantees.

Limitations. Generative evaluation demands substantially higher computational resources ($\sim 3\times$ inference time vs. discrimination mode) and introduces additional variability through decoding strategies. While we use greedy decoding to minimize stochasticity, sampling-based methods (beam search, nucleus sampling) may yield different performance characteristics.

A.6. Training Stability Across Random Seeds

Setup. To validate the reproducibility and stability of HERALD’s training process, we conduct multiple training runs with different random seed initializations. We train with four different random seeds (`{3407, 42, 1234, 9876}`) while keeping all other hyperparameters constant. We report the baseline (plaintext, no encryption) performance to establish training variance bounds without the confounding effects of cryptographic methods.

Findings. Table 11 presents accuracy results across four independent training runs for multiple configurations spanning baseline (plaintext), fully secured (FS+OU), and HERALD (PS+OU) approaches with three representative cryptographic

Configuration	Method	EM↑	BLEU↑	R1↑	R2↑	RL↑	BERT-P↑	BERT-R↑	BERT-F1↑
Baseline	Plaintext	73.8	82.1	85.3	79.5	83.7	89.3	88.5	88.9
FS + OU	AES (SIV)	48.9	56.6	63.6	51.4	60.5	72.8	70.3	71.5
	AES (ECB)	47.6	55.1	62.3	50.0	59.2	71.9	69.4	70.6
	Blowfish (ECB)	45.8	53.2	60.8	48.3	57.8	70.8	68.3	69.5
	FPE	49.3	57.0	64.0	51.8	60.9	73.2	70.7	71.9
FS + OFS	AES (SIV)	41.1	49.9	58.0	45.2	54.8	67.4	64.9	66.1
	AES (ECB)	39.8	48.4	56.7	43.8	53.5	66.5	64.0	65.2
	Blowfish (ECB)	37.0	46.2	54.8	41.7	51.8	65.2	62.6	63.9
	FPE	41.5	50.3	58.4	45.6	55.2	67.8	65.3	66.5
HERALD (PS+OU)	AES (SIV)	64.3	71.6	78.0	67.6	75.7	83.7	81.3	82.5
	AES (ECB)	62.9	70.0	76.7	66.1	74.4	82.9	80.6	81.7
	Blowfish (ECB)	62.5	69.6	76.4	65.7	74.1	82.6	80.3	81.4
	FPE	64.7	72.0	78.4	68.0	76.1	84.1	81.7	82.8
HERALD (PS+OPS)	AES (SIV)	59.6	66.1	73.8	61.6	70.6	80.5	78.4	79.6
	AES (ECB)	58.3	64.7	72.6	60.2	69.4	79.6	77.4	78.5
	Blowfish (ECB)	57.2	63.6	71.8	59.1	68.6	79.1	76.9	78.0
	FPE	60.0	66.5	74.2	62.0	71.0	80.9	78.8	80.0

Table 10. **Generative MCQ performance across encryption configurations and cryptographic methods.** Models generate full option text evaluated using exact match (EM), BLEU, ROUGE (R1/R2/RL), and BERTScore (P/R/F1). All metrics reported as percentages. ↑ indicates higher is better. Baseline uses no encryption (plaintext). Best performing encryption method per configuration in **bold**.

methods. The baseline achieves highly consistent performance: mean accuracy of $76.53 \pm 0.19\%$ (standard deviation: 0.19 percentage points) across the four seeds. Encrypted configurations exhibit slightly higher variance but remain stable: HERALD PS+OU with FPE shows $68.25 \pm 0.25\%$, AES (SIV) achieves $67.24 \pm 0.23\%$, and Blowfish (ECB) yields $66.98 \pm 0.30\%$. Fully secured configurations (FS+OU) demonstrate comparable stability: FPE at $53.11 \pm 0.24\%$, AES (SIV) at $53.48 \pm 0.21\%$, and Blowfish (ECB) at $52.86 \pm 0.28\%$. The coefficient of variation remains below 0.6% across all configurations, demonstrating robust training convergence. Critically, the observed performance gaps between encryption approaches (e.g., 15.1 percentage points between HERALD PS+OU FPE and FS+OU FPE) substantially exceed the maximum standard deviation (0.30 points), confirming that architectural differences drive performance variation rather than training stochasticity.

Implications. The consistent variance patterns across seeds confirm that HERALD’s utility-privacy trade-offs are not artifacts of fortuitous initialization. Standard deviations ranging from 0.19% (baseline) to 0.30% (encrypted configurations) are negligible compared to the 15–20 percentage point performance gaps between plaintext baseline and fully secured configurations, or the 14–15 point improvements HERALD achieves over full encryption approaches. This sta-

bility validates our use of single-seed experiments for computational efficiency is justified given the low inherent variance ($CV < 0.6\%$). Notably, encrypted configurations exhibit marginally higher variance than baseline (0.21–0.30% vs. 0.19%), likely reflecting additional stochasticity introduced by encrypted token representations, yet this remains well within acceptable bounds.

Limitations and Computational Constraints. While we establish training stability for baseline configurations, we do not conduct exhaustive multi-seed analysis across all 24 encryption configurations (6 cryptographic methods $\times$ 2 security levels $\times$ 2 option encryption strategies) due to prohibitive computational costs. Given the demonstrated stability of baseline training and the large effect sizes in our comparative analyses, we prioritize breadth of cryptographic exploration over seed replication.

A.7. Cryptography and Key Management

This section expands the cryptographic key choices used by HERALD, emphasizing practical implications for model training and deployment.

A.7.1. Key Size and Ciphertext Footprint

Setup. We quantify how *key size* and *cipher mode* shape (i) ciphertext string length at the token level, (ii) sequence-

Configuration	Method	Seed 3407	Seed 42	Seed 1234	Seed 9876	Mean $\pm$ Std	CV (%)
Baseline	Plaintext	76.51	76.31	76.78	76.51	76.53 $\pm$ 0.19	0.25
FS + OU	AES (SIV)	53.47	53.28	53.76	53.39	53.48 $\pm$ 0.21	0.38
	Blowfish (ECB)	52.84	52.51	53.19	52.89	52.86 $\pm$ 0.28	0.53
	FPE	53.10	52.84	53.42	53.06	53.11 $\pm$ 0.24	0.45
HERALD (PS+OU)	AES (SIV)	67.23	66.94	67.49	67.28	67.24 $\pm$ 0.23	0.34
	Blowfish (ECB)	66.99	66.61	67.34	66.97	66.98 $\pm$ 0.30	0.45
	FPE	68.25	67.98	68.58	68.17	68.25 $\pm$ 0.25	0.37

Table 11. **Training stability analysis across four random seeds.** Results span baseline (plaintext), fully secured (FS+OU), and HERALD (PS+OU) configurations with three representative cryptographic methods (AES (SIV), Blowfish (ECB), FPE). All configurations exhibit low variance (coefficient of variation $< 0.60\%$), with standard deviations ranging from 0.19–0.30 percentage points. **CV** = Coefficient of Variation.

length inflation after tokenization, and (iii) task utility. We use the HERALD partially-secured setting with options left in plaintext (PS+OU). Table 12 reports, for each cipher variant, the average input tokens, inflation relative to plaintext, ciphertext characters per secured token, and utility metrics (Accuracy and F_1 -score). Figure 7 gives a bar plot comparison of sequence-length inflation relative to plaintext.

Findings. *Mode dominates footprint.* AES (SIV) inflates inputs by 51.0–51.7%, FPE (AES-128) by 57.0%, while ECB-based schemes expand sequences far more, $\approx 132.3\%$ (Blowfish-ECB) and 179.5–179.8% (AES-ECB). Ciphertext verbosity tracks this trend (mean chars per secured token: ~ 3.8 for SIV, 6.45 for FPE (AES-128), 16.1 for Blowfish-ECB, 24.1 for AES-ECB). *Utility declines relative to plaintext* (76.58% $\rightarrow$ 65–68%), but remains competitive for deterministic schemes: FPE (AES-128) attains the highest accuracy (68.23%), closely followed by AES-128-SIV (68.03%) at materially lower overhead (51.0% vs. 57.0%). *Key size has slight effect within a mode:* moving from 128 to 256 bits (for AES-SIV) changes inflation by ≤ 0.7 pp (which may be due to conversion to the UTF-8 encoding) and accuracy by ≤ 2.0 pp (e.g., AES-128-ECB 67.22% vs. AES-256-ECB 65.23%; AES-128-SIV 68.03% vs. AES-256-SIV 67.13%). Overall, SIV and FPE (AES-128) lie on the Pareto frontier (best accuracy–overhead trade-off), whereas ECB variants are substantially more verbose without commensurate utility gains.

A.8. Efficiency, Resource Use, and Constraints

We report practical costs of HERALD’s preprocessing and the runtime effects of ciphertext/marker sequences, relative to plaintext baselines.

Variant	Avg. tokens	Infl. (%)	Chars/ct (mean)	Acc. (%)	F_1 -score (%)
Plaintext	81.09	0.00	0.00	76.58	76.42
AES-128-SIV	122.46	51.01	3.77	68.03	67.97
AES-256-SIV	123.02	51.70	3.81	67.13	66.57
FF1-FPE (AES-128)	127.33	57.02	6.45	68.23	68.05
Blowfish-ECB	188.34	132.25	16.13	66.83	66.76
AES-128-ECB	226.68	179.53	24.12	67.22	66.59
AES-256-ECB	226.89	179.79	24.12	65.23	65.11

Table 12. **Key size & mode vs. ciphertext footprint and utility.** Avg. *tokens* is the mean tokenized input length under the model’s tokenizer; *Infl. (%)* is relative to plaintext; *Chars/ct* measures average characters inside markers per secured token (proxy for ciphertext verbosity). Accuracies/ F_1 -score are exact set-wise metrics.

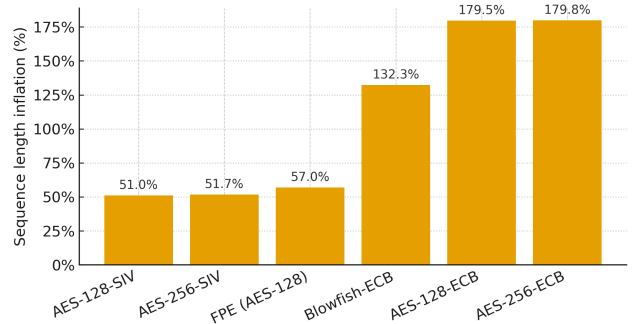


Figure 7. **Sequence-length inflation relative to plaintext.** Lower is better.

A.8.1. Encryption Throughput and Preprocessing Time

Setup. We benchmark per-method *conversion speed* (ms/token and MB/s) in Table 13 and *preprocessing* latency in Table 14 for the HERALD pipeline. Conversions are measured on a flat stream of 40,000 short cleaned tokens; preprocessing times average over 200 short texts. We report results for reversible ciphers (AES-SIV, AES-ECB, Blowfish-ECB), format-preserving encryption (FPE), similarity-preserving

transforms (Fuzzy hash, Soft Hash), and one-way cryptographic hashes (MD5, SHA-1, SHA-256), together with the baseline: Plaintext/Baseline-IDENTITY (identity mapping). We additionally evaluate a *batch/offline* strategy that memorizes token→ciphertext lookups (“cache”), reflecting deployments that pre-secure a sensitive lexicon. Design choices and markers follow the main text.

Findings. (i) *Primitive cost dominates.* Among reversible ciphers, AES (ECB) is fastest (0.0053 ms/token), followed by Blowfish (ECB) (0.0075) and AES (SIV) (0.0091). One-way hashes are substantially faster (MD5 $\approx$ 0.0010, SHA-1 $\approx$ 0.00105, SHA-256 $\approx$ 0.00110 ms/token), while FPE (0.0088) and Soft Hash (0.0083) are slower than hashes but faster than AES (SIV). (ii) *Caching usually helps across the board.* Offline memorization improves throughput by $\sim 3\text{--}10\times$ depending on method (e.g., AES-ECB $\rightarrow$ 0.0012 ms/token; MD5 $\rightarrow$ 0.00030). (iii) *Pipeline overhead is small and stable.* Preprocessing remains ≈ 0.02 ms/example for all secured methods (not including the time required by other steps in HERALD); plaintext pass-through is ≈ 0.0002 ms/example. Input/output MB/s remain flat across methods. Overall, securing methods adds acceptable latency relative to model inference, and batch/offline strategies provide predictable wins for high-reuse vocabularies.

Notes. Plaintext and Baseline-IDENTITY both denote identity mappings (minor numeric differences arise from harness effects). “Cache” denotes token→ciphertext memoization simulating offline pre-encryption.

Method	ms/token (no cache)	MB/s (no cache)	ms/token (cache)	MB/s (cache)
Baseline-IDENTITY	0.0001003	51.8320	0.0001457	35.6976
Plaintext	0.0001067	48.7472	0.0001436	36.2072
AES (SIV)	0.0091485	0.5680	0.0015246	3.4100
AES (ECB)	0.0052602	0.9806	0.0012172	4.2715
Blowfish (ECB)	0.0074852	0.6892	0.0014415	3.5776
FPE	0.0088391	0.5871	0.0018553	2.8079
Fuzzy Hash	0.0016508	3.1496	0.0004081	12.7420
Soft Hash	0.0083077	0.6172	0.0016515	3.1980
MD5	0.0010063	5.1669	0.0003023	17.2013
SHA-1	0.0010496	4.9538	0.0003115	16.6934
SHA-256	0.0011000	4.7268	0.0003337	15.5823

Table 13. Per-method **conversion speed** on a 40k-token stream. Lower ms/token is better; higher MB/s is better. “Cache” denotes offline memoization.

Interpretation. For deployments desiring reversible protection with maximum throughput, AES (ECB) (deterministic) offers the best speed; for irreversible obfuscation, MD5/SHA-1/SHA-256 provide the strongest throughput, with SHA-256 preferred when higher assurance is required. Offline pre-encryption (caching) yields consistent throughput gains and is recommended when sensitive vocabularies recur frequently.

Method	Cache	ms/example	Input MB/s	Output MB/s	Examples
Plaintext	×	0.0002619	379.7640	379.7640	200
Plaintext	✓	0.0002263	439.4755	439.4755	200
AES (SIV)	×	0.0196729	5.0549	5.1459	200
AES (SIV)	✓	0.0197841	5.0265	5.1170	200
AES (ECB)	×	0.0206570	4.8141	4.9008	200
AES (ECB)	✓	0.0200035	4.9714	5.0609	200
Blowfish (ECB)	×	0.0205607	4.8366	4.9237	200
Blowfish (ECB)	✓	0.0200097	4.9698	5.0593	200
FPE	×	0.0201783	4.9283	5.0170	200
FPE	✓	0.0201679	4.9309	5.0196	200
Fuzzy Hash	×	0.0200703	4.9548	5.0440	200
Fuzzy Hash	✓	0.0200034	4.9714	5.0609	200
Soft Hash	×	0.0205573	4.8374	4.9245	200
Soft Hash	✓	0.0204300	4.8676	4.9552	200
MD5	×	0.0203614	4.8840	4.9719	200
MD5	✓	0.0198677	5.0054	5.0954	200
SHA-1	×	0.0223959	4.4403	4.5202	200
SHA-1	✓	0.0202946	4.9001	4.9883	200
SHA-256	×	0.0198673	5.0055	5.0956	200
SHA-256	✓	0.0202859	4.9022	4.9904	200

Table 14. **Preprocessing** for all secured methods. Lower ms/example is better; higher MB/s is better. $n=200$ examples. Results are shown with cache (✓) and without cache (×).

A.8.2. Sequence Length Inflation and Memory

We quantify how HERALD’s ciphertext and boundary markers ($\#^{\wedge} \dots \#^{\wedge}$) affect tokenization and memory relative to plaintext. We evaluate seven controlled variants: plaintext; fully secured stem with options unsecured, without/with markers (FS+OU w/o, w/); HERALD-partially secured stem with options unsecured, without/with markers (PS+OU w/o, w/); and securing the options as well—PS+OPS and FS+OFS (both with markers). Tokens are counted post-template; batch size at fixed memory is estimated as the plaintext batch divided by the inflation ratio (Table 15); we also measure peak CUDA memory on a single forward pass (Table 16). Delimiter design and the FS/PS and OU/OFS/OPS settings follow our framework description.

Findings. (i) *Ciphertext alone* causes modest growth: FS+OU (w/o markers) and PS+OU (w/o markers) inflate tokens by +22.6% and +24.7%, reducing the feasible batch from 4→3 at fixed memory. (ii) *Markers dominate* inflation: adding markers on the stem lifts inflation to +63.6% (PS+OU w/) and +85.3% (FS+OU w/), cutting batch to 2. (iii) *Securing options* multiplies encrypted spans: PS+OPS reaches +143.2% and FS+OFS +178.5%, forcing batch 1. (iv) Empirical GPU measurements usually track token growth: peak memory rises from 6.84 GB (plaintext) to 9.15–10.61 GB for marked stems and 10.81 GB for FS+OFS (+51%). Overall, delimiter count—hence the number of secured spans—drives both sequence length and memory. In practice, PS+OU (w/ markers) balances privacy with tractable memory; omitting markers trims length but loses explicit boundaries that stabilize tokenization.

Variant	Mean Tokens	Std Tokens	Inflation vs. Plain (%)	Batch @ same memory (est.)
Plaintext	60.30	26.65	0.00	4
FS+OU (no markers)	73.95	44.81	22.64	3
FS+OU (with markers)	111.76	91.34	85.33	2
FS+OFS (with markers)	167.94	105.65	178.50	1
PS+OU (no markers)	75.21	45.38	24.72	3
PS+OU (with markers)	98.63	78.12	63.57	2
PS+OPS (with markers)	146.66	88.64	143.22	1

Table 15. **Token inflation and batch-size implications** at fixed memory budget. Inflation is relative to plaintext. OU: options unsecured; OFS/OPS: options fully/partially secured.

Variant	Peak Bytes	Peak MiB
Plaintext	7,174,982,144	6,842.60
FS+OU (no markers)	8,246,117,888	7,864.11
FS+OU (with markers)	9,595,937,280	9,151.40
FS+OFS (with markers)	10,815,572,480	10,314.53
PS+OU (no markers)	9,703,117,312	9,253.61
PS+OU (with markers)	10,606,887,424	10,115.52
PS+OPS (with markers)	10,408,585,728	9,926.40

Table 16. Empirical **peak CUDA memory** per variant (single forward pass, same batch across rows).

Implications. Because memory scales roughly with effective sequence length in decoder-only LMs, privacy settings should be chosen with hardware in mind: encrypting only the stem (OU) is notably cheaper than also securing options; reducing the number of secured spans (and thus markers) is the most direct lever for keeping batch sizes practical while retaining HERALD’s protection boundaries.

B. Ablations and Privacy–Utility Trade-offs

Here we isolate the contribution of each HERALD component and quantify operating points on the privacy–utility frontier.

B.1. Privacy Levels in HERALD

We define Partial-A/B/C/D/E policy tiers. These policies are instantiated by POS-driven coverage rules.

B.1.1. POS-Selective Encryption

We evaluate the impact of part-of-speech (POS) selective encryption on task performance by progressively encrypting different (and more) grammatical categories. This analysis examines the trade-off between privacy protection and model utility when encrypting content words versus function words.

Setup. We implement five encryption levels, each targeting specific POS categories:

- **Level A:** NOUN only

- **Level B:** NOUN + PROPN (proper nouns)
- **Level C:** NOUN + PROPN + ADJ (adjectives)
- **Level D:** NOUN + PROPN + ADJ + VERB
- **Level E:** NOUN + PROPN + ADJ + VERB + ADV (adverbs)

Tokens matching the target POS categories are encrypted using AES (SIV) and wrapped with markers, while function words and other categories remain in plaintext.

Findings. Table 17 presents the evaluation results across all encryption levels. Performance degrades monotonically as more POS categories are encrypted, with the steepest decline occurring when verbs are added (Level D). Encrypting only nouns (Level A) preserves 70.15% accuracy, while comprehensive content word encryption (Level E) reduces performance to 61.28%, representing a 8.87 percentage point decrease from the most conservative setting.

The results demonstrate that nouns and proper nouns carry substantial semantic information for medical question answering, as their encryption (Levels A-B) causes relatively modest performance degradation (0.40 percentage points). However, verbs appear particularly critical, with their inclusion (Level D) causing the largest single-step performance drop (3.45 percentage points). This suggests that action and process information encoded in verbs is essential for strong performance in HERALD.

Level	POS Categories	Accuracy	F ₁
A	NOUN	0.7015	0.7014
B	NOUN + PROPN	0.6975	0.6972
C	NOUN + PROPN + ADJ	0.6812	0.6808
D	NOUN + PROPN + ADJ + VERB	0.6467	0.6463
E	NOUN + PROPN + ADJ + VERB + ADV	0.6128	0.6124

Table 17. **Performance impact of POS-selective encryption.** Results show accuracy and F₁-score across five encryption levels targeting different grammatical categories.

Implications. The preservation of function words (articles, prepositions, conjunctions) maintains syntactic structure while protecting the most semantically dense tokens. This selective approach offers a practical middle ground between full encryption (maximum privacy, poor utility) and no encryption (maximum utility, no privacy).

Content words (e.g. nouns) typically carry domain-specific knowledge essential for specialized tasks like medical QA, while function words primarily provide grammatical scaffolding. Our results quantify this trade-off empirically, suggesting that noun and proper noun (PROPN) encryption (Level B) may offer an optimal privacy-utility balance for many applications.

B.2. Marker Scheme Variants for Secured Tokens

Setup. We evaluate eight distinct marker schemes for delimiting encrypted tokens within the input text to understand their impact on model learnability and privacy preservation. The baseline approach embeds encrypted tokens directly without delimiters, while seven alternative schemes employ various sentinel token patterns to explicitly mark encrypted content boundaries.

Findings. Table 18 presents the performance comparison across all marker schemes. The scheme utilized in our main study ($\#^{\wedge}\dots^{\wedge}\#$) achieves the highest accuracy (68.54%), followed closely by mathematical notation delimiters ($\$^{\wedge}\dots^{\wedge}\$$) at 67.94%. The no-marker baseline achieves 63.43% accuracy, indicating that while models can learn to distinguish encrypted content without explicit delimiters, marker schemes provide meaningful performance improvements. The marker schemes exhibit a performance range of 5.11 percentage points (63.43% to 68.54%), indicating that delimiter choice has a measurable impact on model performance.

Implications. The superior performance of the marker scheme utilized in our study ($\#^{\wedge}\dots^{\wedge}\#$) validates our methodological choice for the main experiments. This scheme’s effectiveness likely stems from its distinctiveness from natural language patterns and clear visual separation of encrypted content. Mathematical notation markers perform comparably well, suggesting that symbolic delimiters provide effective semantic boundaries for model learning. The moderate performance gap between marked and unmarked approaches (5.11 percentage points) demonstrates that while encrypted token distributions are learnable without delimiters, explicit markers provide substantial improvements in model comprehension.

Scheme	Marker Pattern	Acc.	F ₁
No markers	<i>Only encrypted tokens</i>	63.43	63.36
Utilized in the study	$\#^{\wedge}\dots^{\wedge}\#$	68.54	68.50
Medical XML	<MED>...</MED>	66.24	66.14
Security brackets	[SEC]...[/SEC]	65.54	65.50
Identity brackets	[ID]...[/ID]	65.54	65.57
Special tokens	< sec >...</sec >	66.25	66.20
Unicode angles	<...>	64.25	64.25
Hash symbols	#...#	66.04	66.95
Math notation	$\$^{\wedge}\dots^{\wedge}\$$	67.94	67.80

Table 18. **Performance comparison of marker schemes** for encrypted token delimitation.

B.3. Stopwords Impact

Setup. We investigated the impact of stopwords removal on performance across different encryption coverage strategies. The experiments compared two preprocessing configurations: (1) retaining stopwords during encryption ($\checkmark$), and (2) removing English stopwords prior to sensitive token identification and encryption ($\times$). Each configuration was evaluated under both partial encryption (HERALD’s standard approach) and full encryption (encrypting all tokens) scenarios.

Findings. Stopword preprocessing demonstrated distinct effects across different encryption configurations and task types (Table 19). In HERALD configurations, stopwords retention consistently outperformed removal across both tasks: for classification, retention achieved 56.7% accuracy (56.2% F₁) compared to 52.8% accuracy (52.5% F₁) with removal, representing a 3.9 percentage point improvement. Similarly, for MCQ tasks, retention yielded 56.6% accuracy (55.4% F₁) versus 51.5% accuracy (51.3% F₁) with removal, showing a 5.1 percentage point advantage. Whereas, full encryption configurations exhibited opposite trends regarding stopwords preprocessing. For classification tasks, stopwords removal achieved superior performance (46.7% accuracy, 46.1% F₁) compared to retention (40.9% accuracy, 40.4% F₁), suggesting that when all tokens are encrypted, removing non-content words may enhance focus on remaining semantic information. MCQ tasks under full encryption showed similar patterns, with removal yielding 37.8% accuracy (29.9% F₁) versus 35.7% accuracy (27.7% F₁) with retention.

Implications. The differential impact of stopwords preprocessing across encryption configurations reveals important insights about the interaction between linguistic preprocessing and encryption strategies. In HERALD’s selective encryption approach, preserving stopwords maintains valuable contextual information that aids model comprehension, as common function words provide syntactic and semantic scaffolding around encrypted medical terms. The consistent 4-5 percentage point advantages of stopwords retention in HERALD suggest that these linguistic markers enhance the model’s ability to interpret encrypted content within preserved structural context. Conversely, under comprehensive encryption where all tokens are obfuscated, removing stopwords appears beneficial, likely because it reduces the overall encrypted token load and may help models focus on the remaining semantic content. This finding indicates that optimal preprocessing strategies are configuration-dependent: selective encryption benefits from linguistic context preservation, while comprehensive encryption may benefit from content filtering.

Task	Configuration	Stopwords	Acc.	F ₁
Classification	Fully Secured	×	0.467	0.461
	Fully Secured	✓	0.409	0.404
	HERALD	×	0.528	0.525
	HERALD	✓	0.567	0.562
MCQ	Fully Secured	×	0.378	0.299
	Fully Secured	✓	0.357	0.277
	HERALD	×	0.515	0.513
	HERALD	✓	0.566	0.554

Table 19. **Performance comparison across stopword preprocessing and encryption coverage configurations.** × indicate stopword removal prior to encryption; ✓ indicates stopword retention.

B.4. Lemmatization Strategy

Setup. We evaluate two distinct lemmatization strategies for text preprocessing: (1) proposed *targeted lemmatization*, which applies lemmatization selectively based on part-of-speech (POS) tagging and named entity recognition (NER), and (2) *untargeted lemmatization*, which applies lemmatization indiscriminately to all non-stop words in the text.

The targeted approach implements a two-stage process: first, POS tagging identifies non-sensitive words that should remain unlemmatized to preserve contextual meaning; second, lemmatization is applied only to words not flagged as contextually critical. This preserves semantic relationships in domain-specific medical terminology while normalizing common linguistic variations. In contrast, the untargeted approach applies lemmatization uniformly across all content words, potentially introducing semantic drift in specialized terminology.

Findings. Experimental results demonstrate that targeted lemmatization strategy as used in HERALD achieves substantial performance improvements across all evaluation metrics compared to untargeted lemmatization (Table 20). The targeted approach yields an accuracy of 69.55% $\pm$ 0.18%, representing a significant 3.40 percentage point improvement over the untargeted baseline (66.15% $\pm$ 0.13%). The targeted strategy shows even more pronounced improvements in F₁-score (69.43% $\pm$ 0.14% vs. 65.79% $\pm$ 0.16%), demonstrating a 3.64 percentage point gain. These results indicate not only enhanced model performance but also improved consistency, as evidenced by the reported standard deviations.

Implications. The substantial performance gains observed with targeted lemmatization strongly validate our hypothesis that selective morphological normalization preserves critical semantic information while reducing lexical diversity. This is particularly crucial in medical question-answering tasks

where precise terminology directly impacts model comprehension and clinical accuracy.

Metric	Untargeted Lemmatization	Targeted Lemmatization
Accuracy (%)	66.15 $\pm$ 0.13	69.55 $\pm$ 0.18
F ₁ -Score (%)	65.79 $\pm$ 0.16	69.43 $\pm$ 0.14
Improvement	Baseline	+3.40% / +3.64%

Table 20. **Performance comparison between targeted and untargeted lemmatization** strategies on medical question answering task. Results show consistent improvements with HERALD’s targeted approach across all evaluation metrics.

B.5. Privacy~Utility Curves

Setup. We perform a parameter sweep over the encryption rate f , defined as the fraction of prompt tokens that are encrypted, with $f \in \{0, 0.10, 0.25, 0.50, 0.75, 1.00\}$. The selection procedure is deterministic: we first encrypt sensitive spans and, as f increases, progressively include non-sensitive spans until the entire prompt is transformed at $f = 1$. Encryption is performed with the HERALD transform (AES-SIV). For each f , we fine-tune and evaluate the same configuration, reporting accuracy on the held-out test split. This realizes the HERALD’s “dial-a-privacy” knob while preserving the training protocol.

Findings. Figure 8 shows a smooth, monotonic trade-off: utility degrades approximately linearly up to $f \approx 0.5$ and more sharply thereafter. Relative to plaintext ($f=0$), fully encrypting all tokens ($f=1$) reduces Accuracy from 76.66% to 53.95%. The *sweet spot* occurs at $f=0.10$, retaining $\sim 97\%$ of baseline accuracy with only a ~ 2.7 -point Accuracy drop (76.66 $\rightarrow$ 73.97). Beyond $f \geq 0.25$, utility declines more noticeably, especially past $f \geq 0.75$.

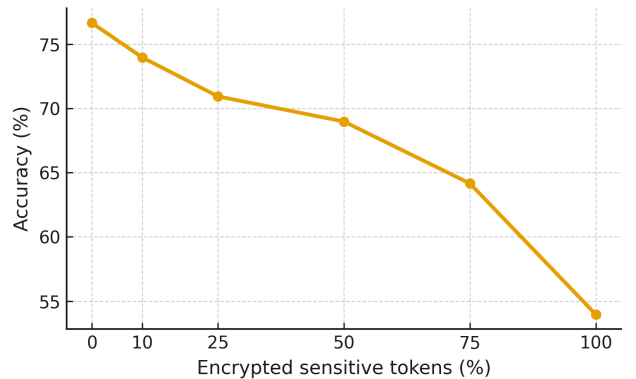


Figure 8. **Privacy~utility curve (Accuracy).** Test Accuracy (%) as a function of the fraction f of tokens encrypted. Accuracy decreases smoothly with f .

C. Security Evaluation and Attack Surface

We map threats to experiments and clarify which protections are offered by HERALD vs. what remains exposed through context.

C.1. Training Data Extraction

Setup. We probe fine-tuned models for regurgitation using four prompts: (i) *verbatim-global-plain* (“repeat a training question verbatim”), (ii) *extraction-list-plain* (“list three training questions verbatim”), (iii) *prefix-continue-plain* (continue a gold question given a long plaintext prefix), and (iv) *prefix-continue-cipher* (HERALD-encoded prefix). We report exact-match rate (EM), mean character-level similarity (MS; 0–1), and the rate at which ciphertext markers ($\#^{\wedge} \dots \wedge^{\#}$) appear in model outputs (CMR). The results are summarized in Table 21.

Findings. (1) Free-form recall fails to extract verbatim text in all cases (EM=0 for both *verbatim-global-plain* and *extraction-list-plain*), but HERALD’s outputs *always* contain ciphertext markers (CMR=1.0), ensuring any recall manifests as ciphertext rather than plaintext. (2) With a long plaintext prefix, both models frequently autocompletes the remainder (EM $\approx$ 0.10–0.65; MS $\approx$ 0.89–0.92) for *both* TRAIN and TEST, indicating anchored next-token completion rather than preferential recall of train items; when continuations touch sensitive content, HERALD flips into ciphertext (CMR $\approx$ 0.43–0.59). This behavior matches HERALD’s design: any memorized sensitive spans, if elicited, appear encrypted. (3) Given a HERALD-encoded prefix, the model never reveals plaintext (EM=0, CMR=1.0) and continues in ciphertext (moderate MS from prompt echo), consistent with the framework’s confidentiality goal.

Implication. Under recall-style prompts, no verbatim training items are recovered, while HERALD routes any residual recall into ciphertext; with strong anchors, both models autocompletes but HERALD preserves confidentiality at sensitive spans.

C.2. Jailbreaking and Prompt Injection

Setup. We evaluate whether HERALD’s token-level cipher spans resist prompt-injection and jailbreak attempts that explicitly solicit decryption or implicitly induce “best-guess” substitutions. The study follows the security evaluation plan with markers introduced in the core workflow to bound cipher spans and stabilize tokenization.

Attack suite. We probe a held-out MCQ subset formatted per the main prompting recipe (question + options + “answer with the letter”). For each secured prompt, we instantiate

Variant	Split	Attack	n	EM (%)	MS	CMR (%)
HERALD	TEST	extraction-list-plain	48	0.0	0.342	100.0
HERALD	TEST	prefix-continue-cipher	48	0.0	0.513	100.0
HERALD	TEST	prefix-continue-plain	48	12.4	0.704	42.5
HERALD	TEST	verbatim-global-plain	48	0.0	0.332	100.0
HERALD	TRAIN	extraction-list-plain	48	0.0	0.326	100.0
HERALD	TRAIN	prefix-continue-cipher	48	0.0	0.539	100.0
HERALD	TRAIN	prefix-continue-plain	48	22.5	0.790	58.8
HERALD	TRAIN	verbatim-global-plain	48	0.0	0.320	100.0
Plaintext	TEST	extraction-list-plain	48	0.0	0.411	0.0
Plaintext	TEST	prefix-continue-plain	48	40.2	0.916	0.0
Plaintext	TEST	verbatim-global-plain	48	0.0	0.421	0.0
Plaintext	TRAIN	extraction-list-plain	48	0.0	0.412	0.0
Plaintext	TRAIN	prefix-continue-plain	48	64.6	0.911	0.0
Plaintext	TRAIN	verbatim-global-plain	48	0.0	0.422	0.0

Table 21. **Regurgitation probes.** EM: exact match; MS: mean similarity (0–1); CMR: ciphertext-marker rate. Free-form recall fails (EM=0), and HERALD always emits ciphertext in such cases (CMR=1.0). Long-prefix continuation yields high EM for both models, including TEST, indicating anchored completion rather than train-specific memorization; HERALD transitions into ciphertext when continuations touch sensitive spans. Values are averaged over multiple runs.

six attacks: DIRECT-DECRYPT-REQUEST, EXPLAIN-THEN-DECRYPT, PRETEND-KEY-AVAILABLE, MARKER-STRIP-REWRITE, SUMMARIZE-REVEAL-SECRETS, and a marker-ablation control NOMARKERS-GUESS-REPLACEMENTS. The first five operate on the original HERALD-protected text; the control removes $\#^{\wedge} / \wedge^{\#}$ before issuing a rewrite/guess instruction.

Protocol and metric. We evaluate $N=512$ secured prompts (deterministic decoding). A response is flagged *explicit-success* if it unambiguously claims decryption or plaintext revelation (e.g., “decrypted: ...”), and *alpha-guess* if it appears to replace a referenced ciphertext span with nearby alphabetic content (heuristic windowing). We report per-attack rates and *any-success* = *explicit* $\vee$ *alpha-guess*. Table 22 summarizes the results.

Attack	Explicit $\downarrow$	Alpha-guess $\downarrow$	Any $\downarrow$
Direct-Decrypt-Request	0.000	0.000000	0.000000
Explain-Then-Decrypt	0.000	0.000000	0.000000
Pretend-Key-Available	0.000	0.000000	0.000000
Summarize-Reveal-Secrets	0.000	0.015625	0.015625
Marker-Strip-Rewrite	0.000	0.046875	0.046875
NoMarkers-Guess-Replacements	0.000	0.000000	0.000000

Table 22. **Heuristic jailbreak/prompt-injection success on HERALD-protected prompts** ($N=512$). “Explicit” denotes overt claims of decryption; “alpha-guess” flags local plaintext-like substitutions near cited ciphertext. All rates are fractions in $[0, 1]$.

Findings. (1) No attack achieved *explicit* decryption disclosure—consistent with the design goal that ciphertext

spans are unrecoverable without the secret key. (2) Low but non-zero *alpha-guess* appears when attacks instruct the model to rewrite while removing markers (MARKER-STRIP-REWRITE, 4.69%) or to summarize then list “secrets” (1.56%), yet absolute rates remain small. (3) The marker-ablation control shows 0% under our heuristic; because detection anchors on nearby ciphertext substrings, this conservative metric likely *undercounts* guesses when markers (and thus the cipher anchor) are absent⁶.

Effect of markers. HERALD’s explicit span delimiters serve two purposes: they prevent tokenizer fragmentation of non-linguistic ciphertext substrings and teach the model a stable “foreign alphabet” for secured spans during fine-tuning, reducing the odds that rewriting instructions are interpreted as a request to infer plaintext. Empirically, attacks that operate *without* respecting markers do not yield overt disclosures, and the observed guess-like behavior stays rare. This aligns with the framework rationale that markers bound the secured region and preserve learnability without exposing content.

C.3. Semantic Similarity Leakage

We assess whether HERALD’s ciphertext tokens carry residual semantics in off-the-shelf embedding space. For each sensitive plaintext token w identified by the HERALD pipeline, we form a paired ciphertext token z using one of nine methods (AES-SIV/ECB, Blowfish-ECB, FPE, Fuzzy Hash, Soft Hash, MD5, SHA-1, SHA-256). We embed w and z with a sentence encoder and compute cosine similarities for aligned (“true”) pairs and for a null baseline obtained by shuffling ciphertexts. We report distributional separation (AUROC/AUPRC), two-sample tests (KS, Mann–Whitney U , Welch t ; Benjamini–Hochberg adjusted q), and a histogram-overlap coefficient.

Findings. Across all 18 runs (9 methods $\times$ with, without markers), the aligned and shuffled cosine distributions are statistically indistinguishable: AUROC $\approx$ 0.49–0.51; KS/MW/ t p -values remain high (BH- $q \approx$ 0.96) for every method/setting; histogram overlaps are large ($\sim$ 0.78–0.84). The largest mean shift observed is $\Delta\mu=0.003$ for FPE (no markers), still practically negligible. Consequently, we find no evidence of embedding-space semantic leakage for any method under this test. Method choice may therefore be guided by reversibility and token-length inflation rather than embedding leakage (Table 23).

Notes on cost. Length inflation varies by method: similarity hashes and FPE are compact (median length ratios

⁶Heuristic limitations: if an attack elicits fluent replacements without echoing the original ciphertext substring, *alpha-guess* may miss it. We therefore interpret the table as a *lower bound* on guess-like leakage.

Method	Family	AUROC $\downarrow$		$ \Delta\mu \times 10^3 \downarrow$		Overlap $\uparrow$		Length ratio $\downarrow$	
		$\checkmark$	$\times$	$\checkmark$	$\times$	$\checkmark$	$\times$	$\checkmark$	$\times$
AES (SIV)	symmetric enc.	0.504	0.497	+2.031	+1.293	0.815	0.782	4.731	4.199
AES (ECB)	symmetric enc.	0.505	0.499	+1.802	+1.426	0.791	0.800	3.519	2.987
Blowfish (ECB)	symmetric enc.	0.498	0.501	−0.718	+0.689	0.807	0.824	2.702	2.170
FPE	format-preserving	0.494	0.507	−1.070	+3.017	0.816	0.791	1.518	0.986
Fuzzy Hash	similarity hash	0.501	0.502	−0.778	−0.966	0.834	0.827	1.601	1.068
Soft Hash	similarity hash	0.505	0.506	+1.558	+1.575	0.814	0.810	2.662	2.130
MD5	crypto hash	0.498	0.498	−0.613	−0.573	0.808	0.802	4.792	4.259
SHA-1	crypto hash	0.503	0.501	+0.961	+1.380	0.799	0.795	5.857	5.324
SHA-256	crypto hash	0.493	0.493	−0.823	−0.533	0.788	0.783	9.051	8.519

Table 23. **Embedding-space leakage check.** For each method we report AUROC (higher worse for leakage), mean-shift $\Delta\mu = \text{mean}*\text{true} - \text{mean}*\text{null}$ (scaled by 10^3), histogram overlap (higher implies more similar distributions), and length ratio (cipher/plain). Results are shown with ($\checkmark$) and without ($\times$) markers. All two-sample tests are non-significant after BH correction (BH- $q \approx$ 0.96 across rows).

$\approx$ 1.0–2.7), symmetric ciphers moderate (2.2–4.7), and cryptographic hashes highest (4.3–9.1). These costs affect context budget but not leakage in our analysis.

Limitations. This check targets type-level similarity of isolated token forms. It does not quantify context-driven inference (where surrounding plaintext could reveal meaning).

Interpretation. Values asymptoting to AUROC $\approx$ 0.50, tiny $\Delta\mu$ (all $|\Delta\mu| \leq 3 \times 10^{-3}$), and large overlaps collectively indicate that embeddings do not align ciphertexts with their plaintext analogs any better than chance. Within this test’s scope, all methods “pass” the semantic-similarity leakage criterion; practitioners can therefore prioritize reversibility (e.g., AES, FPE), digest compactness (e.g., FPE), or strong one-wayness (e.g., SHA-256) without sacrificing this privacy property.

C.4. Embedding Space Probing

Goal. We test whether secured (cipher) tokens inherit distributional structure from their plaintext counterparts in a standard embedding space. If alignment exists, cipher tokens should (i) retrieve the same neighbors as their plaintext tokens and (ii) form clusters consistent with plaintext semantics.

Setup. From frequent sensitive tokens, we built a 100-token probe set per method and produced cipher tokens using the project’s implementations (AES-SIV, Blowfish-ECB, and FPE). We embedded short, neutral contexts containing either the plaintext or the delimiter-wrapped cipher token ($\#^{\wedge} \dots^{\wedge} \#$). Metrics: (1) k -NN set overlap at $k=5$ between plaintext and cipher neighborhoods (cosine); (2) Spearman correlation ρ between pairwise plaintext and cipher cosine similarities; (3) k -means ($k=10$) qualitative inspection; (4) 2-D projections (UMAP, t-SNE) with trustworthiness and a permutation baseline for the k -NN-overlap statistic.

Method	n	kNN@5	ρ (cos)	UMAP kNN@5	t-SNE kNN@5	Trustw. (U)	Trustw. (T)	p (U/T)
AES (SIV)	100	0.019	-0.012	0.039	0.033	0.860	0.889	0.088 / 0.253
Blowfish (ECB)	100	0.035	-0.153	0.033	0.036	0.874	0.888	0.265 / 0.170
FPE	100	0.045	+0.023	0.022	0.031	0.885	0.896	0.798 / 0.379

Table 24. **Embedding-space probing of cipher tokens.** k NN@5: plaintext-cipher neighbor-set overlap (higher $\uparrow$ better); ρ : Spearman correlation of pairwise cosine similarities between plaintext and cipher spaces (higher $\uparrow$ better). UMAP/t-SNE columns report 2-D overlaps; “Trustw.” is trustworthiness (higher $\uparrow$ better). p -values compare observed 2-D overlaps to a permutation baseline; none are significant at $\alpha=0.05$.

Findings. Across methods, neighborhood overlap is small and near permutation baselines, and ρ is near-zero or negative (Table 24). AES (SIV) shows a weak trend in UMAP space (overlap 0.039, $p \approx 0.088$) but remains non-significant. Blowfish (ECB) exhibits the strongest anti-correlation ($\rho = -0.153$), while FPE is closest to chance ($\rho = 0.023$; mixed 2-D overlaps and the largest p -values). k -means partitions of cipher embeddings were heterogeneous and did not recover plaintext semantic groupings (e.g., clusters mixing *patient*, *blood*, *fever*, *management*). Overall, we find no statistically reliable evidence that cipher tokens align to plaintext semantics under these securing schemes.

C.5. Recoverability of Tokens

Setup. We evaluate whether a plaintext token can be “recovered” from the embedding space by nearest-neighbor (NN) search against its own transformed counterpart. Concretely, for a vocabulary subset we compute input-embedding centroids of (i) the plaintext token and (ii) the corresponding secured string $E_k(\cdot)$. We measure pairwise Euclidean distances (primary metric) from plaintext to all secured embeddings and report the exact-match rate that the gold secured token is the top neighbor (NN@1) or among the top five (NN@5). To probe the role of markers, we consider two embedding variants: (a) excluding marker subtokens when forming centroids; (b) including markers, reflecting what the model actually “sees.” This aligns with HERALD’s deterministic token-level transform and marker design.

Results. Table 25 reports Euclidean Top- k recoverability (3000 tokens). Table 26 expands to multiple distances under both marker-handling regimes (5000 tokens). Across seven transforms (AES-SIV, AES-ECB, Blowfish-ECB, FPE, Fuzzy Hash, MD5, SHA-1), NN@1 and NN@5 are essentially at chance: all accuracies are $\leq 0.17\%$, indicating negligible recoverability via nearest neighbors in the model’s input-embedding space. Including markers generally further suppresses NN@1. Alternative distances (cosine, dot-product, Mahalanobis/whitened, CSLs) yield similarly near-zero NN@1, with minor fluctuations that do not change the conclusion. These observations are consistent with HERALD’s goal: deterministic, delimiter-bounded cipher-

text token embeddings behave as distinct token types whose vectors do not systematically align with their plaintext counterparts in embedding space (i.e., they do not geometrically ‘point back’ to plaintext).

Transform	NN@1	NN@5
AES (SIV)	0.0005	0.0015
AES (ECB)	0.0000	0.0035
Blowfish (ECB)	0.0005	0.0020
FPE	0.0005	0.0020
Fuzzy Hash	0.0005	0.0025
MD5	0.0005	0.0025
SHA-1	0.0005	0.0025

Table 25. **Recoverability via Euclidean distance:** Top- k exact-match accuracy. $N = 3000$ plaintext tokens. Values are averaged over multiple runs.

Transform	Exclude markers					Include markers				
	Euclid.	Cosine	Dot	Mahalanobis	CSLS	Euclid.	Cosine	Dot	Mahalanobis	CSLS
AES (SIV)	0.0003	0.0000	0.0003	0.0003	0.0000	0.0000	0.0000	0.0000	0.0003	0.0000
AES (ECB)	0.0007	0.0010	0.0003	0.0000	0.0000	0.0003	0.0017	0.0000	0.0000	0.0000
Blowfish (ECB)	0.0003	0.0007	0.0003	0.0003	0.0003	0.0003	0.0003	0.0007	0.0000	0.0003
FPE	0.0000	0.0010	0.0000	0.0000	0.0000	0.0003	0.0000	0.0003	0.0000	0.0000
Fuzzy Hash	0.0003	0.0000	0.0000	0.0003	0.0007	0.0000	0.0010	0.0007	0.0003	0.0007
MD5	0.0003	0.0007	0.0000	0.0003	0.0003	0.0007	0.0000	0.0000	0.0007	0.0000
SHA-1	0.0007	0.0007	0.0007	0.0003	0.0010	0.0007	0.0007	0.0007	0.0003	0.0013

Table 26. **Nearest-neighbor accuracy (NN@1) by distance and marker handling.** Left block excludes marker subtokens when embedding secured strings; right block includes them. $N = 5000$. Values are averaged over multiple runs.

Interpretation. All methods produce near-chance NN recoverability under Euclidean distance, and alternative distances do not materially increase it; if anything, markers reduce residual matches further by injecting consistent boundary cues around ciphertext spans. Together with the deterministic token-level design, this suggests that HERALD secures sensitive tokens without leaving an “embedding fingerprint” that trivially links ciphertext to plaintext via nearest-neighbor geometry.